Tworzenie aplikacji z użyciem React

Mateusz Jabłoński

21.10.2026 19:17

Kim jestem?

- programista od 2011 roku
- głównie: Javascript / Typescript / Java / dawniej: PHP
- szkoleniowiec / trener / mentor od 2016 roku
- prywatnie mąż i tata
- ostatnio również aktor w lokalnym teatrze i gracz papierowych RPG

sages

Zaufaj naszemu doświadczeniu

18 lat
na rynku usług IT

500+
klientów biznesowych

29 560+
przeszkolonych osób

4 877+
zorganizowanych szkoleń i warsztatów

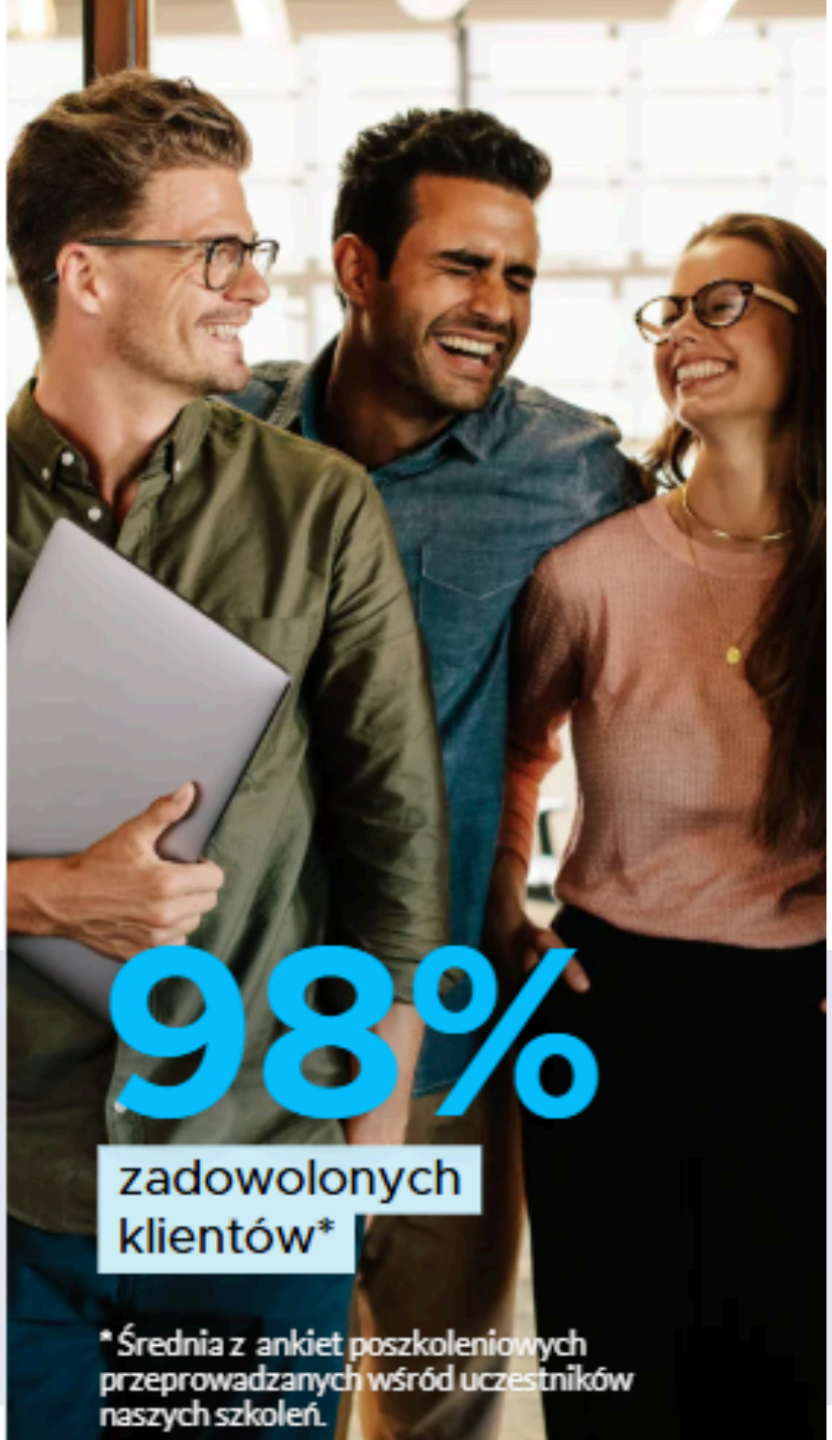
GWARANCJA JAKOŚCI USŁUG

Diamenty Wieloczenka Forbes 2023

ISO 9001:2015 CERTIFIED

STANDARD USŁUGI SZKOLENIOWEJ

Baza Usług Rozwojowych



21.10.2026 19:17

Edukacja na najwyższym poziomie

Wiedza specjalistyczna dla branży IT

Oferujemy szeroki katalog szkoleń z technologii mainstreamowych i specjalistycznych, wschodzących i legacy. Zajęcia prowadzimy w trybie warsztatowym, a programy są oparte o praktyczne know-how. Specjalizujemy się w prowadzeniu dedykowanych szkoleń technologicznych, których agendę dostosowujemy do potrzeb naszych klientów i oczekiwań uczestników.

Wybitni eksperci

Od początku naszego istnienia przeszkoliliśmy dziesiątki tysięcy osób, co pomogło absolwentom podnieść konkurencyjność na rynku pracy i jakość projektów realizowanych na co dzień. Nasze szkolenia prowadzą najlepsi trenerzy, a nasze produkty są oparte na najnowocześniejszej technologii. Ich niezawodność i dopasowanie do potrzeb klientów są możliwe dzięki zespołowi składającemu się z wybitnych ekspertów i ekspertek, którzy/e są na pierwszej linii teorii i praktyki tworzenia i wdrażania innowacji technologicznych.

NASZE POZOSTAŁE MARKI EDUKACYJNE

STACJA.IT

kodoamacz by sages

sages

Najlepsze standardy usług edukacyjnych

Stosujemy Standard Usługi Szkoleniowej Polskiej Izby Firm Szkoleniowych, a nasze usługi realizowane są na najwyższym poziomie, o czym świadczą stale powracający klienci oraz wdrożony certyfikat ISO 9001. Metodologia prowadzonych przez nas zajęć oparta jest na współczesnych narzędziach i dostosowana do potrzeb i oczekiwań klientów.

Ponadto jesteśmy firmą wpisaną do rejestru instytucji szkoleniowych w Wojewódzkim Urzędzie Pracy w Warszawie pod nr 2.14/00133/2019.

Zaufali nam najlepsi

Wśród naszych klientów są takie firmy jak Alior Bank, OLX Group, Bank Zachodni WBK, Orange Polska, Lufthansa i wiele innych.

STUDIA PODYPLOMOWE

Wspieramy organizację zaawansowanych kierunków studiów podyplomowych. Realizujemy zajęcia na kierunkach: Data Science, Big Data i Wizualna analityka danych, AI & Data Driven Business oraz User Experience Design – projektowanie doświadczeń cyfrowych.

Instytut Informatyki
Wydział Elektroniki i Technik Informacyjnych
Politechniki Warszawskiej

AKADEMIA LEONA KOŹMIŃSKIEGO

Uwaga

Wszelkie materiały (treści tekstowe, wideo, ilustracje, zdjęcia itp.) wchodzące w skład szkoleń, kursów i webinarów organizowanych przez Sages są objęte prawem autorskim i podlegają ochronie na mocy Ustawy o prawie autorskim i prawach pokrewnych z dnia 4 lutego 1994 r. (tekst ujednolicony: Dz.U. 2006 nr 90 poz. 631). Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Ustalenia

- Cel i agenda
- Wzajemne oczekiwania
- Pytania i dyskusje
- Elastyczność
- Otwartość i uczciwość

Agenda, czyli co nas czeka?

1. Wprowadzenie
2. Podstawy React
3. React Hooks
4. React w praktyce
5. Zaawansowany React
6. Praca z wybranym mechanizmem zarządzania stanem aplikacji
7. Testowanie (opcja)

github.com/matwjablonski/react-open-10-26

React

React na tle innych rozwiązań

Rozwiązanie	Charakter	Typowy wybór
Angular	kompletny framework	duże aplikacje z silnymi konwencjami
React	biblioteka UI + ekosystem	aplikacje, w których chcemy dobrać architekturę
Vue	framework progresywny	szybki start i stopniowa adopcja
Svelte	kompilator UI	mały runtime i prosty model komponentów

React to biblioteka UI

- React odpowiada za opis komponentów i aktualizację interfejsu
- routing, data fetching, formularze, testy i build są dostarczane przez ekosystem
- dlatego aplikacja React zwykle składa się z Reacta oraz wybranego zestawu narzędzi
- frameworki Reactowe dodają konwencje aplikacyjne ponad samym Reactem

Kiedy biblioteka, kiedy framework?

Podejście	Kiedy ma sens
Vite + React	SPA, panel administracyjny, aplikacja wewnętrzna, nauka Reacta
React Router framework	routing, loadery, akcje i bardziej kompletna architektura SPA
Next.js	SSR, Server Components, Server Actions, SEO, aplikacje full-stack
Remix / inne frameworki	silna praca z formularzami, routingiem i danymi po stronie serwera

Transpilacja

Transpilacja to proces przepisywania kodu do jego odpowiednika w innym języku (lub w tym samym, ale w innej wersji)

Przykłady narzędzi: Babel, TypeScript, SWC (Speedy Web Compiler)

Dlaczego transpilacja jest potrzebna?

- nie wszystkie przeglądarki obsługują najnowsze standardy ECMAScript
- nowe funkcje JS (np. `optional chaining`, `nullish coalescing`, `BigInt`) mogą nie działać w starszych przeglądarkach
- transpilacja pozwala zachować kompatybilność wsteczną

Kod ES6+:

Przykładowy kod:

```
const greet = (name) => `Hello, ${name}!`;
```

Transpilowany kod ES5:

Przykładowy kod:

```
var greet = function(name) {
  return 'Hello, ' + name + '!';
};
```

Polyfills vs Transpilacja

- **Transpilacja** zmienia składnię kodu, aby była zgodna ze starszymi wersjami JS
- **Polyfills** dodają brakujące funkcje do środowiska wykonawczego

Przykładowy kod:

```
// Polyfill dla Array.from
if (!Array.from) {
  Array.from = function(iterable) {
    return [].slice.call(iterable);
  };
}
```

Automatyzacja

- bundlery i narzędzia takie jak `Webpack`, `Vite`, `Parcel` integrują transpilację i polyfille automatycznie
- możemy ustawić docelowe środowisko (targets) w Babel, np.:

Przykładowy kod:

```
{
  "presets": [
    ["@babel/preset-env", {
      "targets": "> 0.25%, not dead"
    }]
  ]
}
```

Dobre praktyki

- transpiluj tylko to, co jest potrzebne (tzw. modern + legacy builds)
- stosuj polyfille tylko tam, gdzie funkcje są naprawdę używane
- regularnie testuj kod w różnych przeglądarkach

Toolchain w aplikacji React

- JSX i TypeScript muszą zostać przetworzone do JavaScriptu zrozumiałego dla przeglądarki
- bundler łączy moduły, CSS i assety w paczki produkcyjne
- dev server zapewnia szybkie odświeżanie aplikacji podczas pracy
- test runner uruchamia testy komponentów, hooków i logiki pomocniczej

Babel, SWC, esbuild

- Babel był przez lata standardowym narzędziem do transformacji JSX i nowego JavaScriptu
- nowoczesne toolchainy często używają szybszych transformacji opartych o SWC albo esbuild
- w szkoleniu nie konfigurujemy transformacji ręcznie - korzystamy z ustawień dostarczonych przez Vite
- ręczna konfiguracja Babel ma sens głównie w istniejących projektach lub niestandardowym buildzie

SPA

- SPA (Single Page Application) aktualizuje widok i obsługuje nawigację bez pełnego przeładowania dokumentu
- w klasycznym wariancie CSR serwer zwraca niewielki dokument HTML, a aplikacja uruchamia się w przeglądarce
- routing i renderowanie widoków odbywają się po stronie klienta, co pozwala zachować stan interfejsu pomiędzy zmianami widoków
- SEO, metadane społecznościowe i szybkość pierwszego renderu wymagają osobnego zaprojektowania
- część kosztu wykonywania i renderowania interfejsu zostaje przeniesiona z serwera na urządzenie użytkownika
- aplikacja może być hostowana jako pliki statyczne, jeśli nie potrzebuje własnego backendu
- podobnie jak MPA, aplikacje SPA mogą być budowane modułowo i komponentowo

Czym jest React?

- biblioteka do budowania interfejsów użytkownika z komponentów
- historycznie React powstał jako odpowiedź na trudność ręcznego utrzymywania i aktualizowania złożonych struktur DOM
- został zaprojektowany i wprowadzony na rynek przez Facebook / Meta
- pozwala opisywać UI deklaratywnie: opisujemy stan docelowy, a React aktualizuje widok
- React opiera się mocno na kompozycji, funkcjach i jednokierunkowym przepływie danych
- React służy do budowania dynamicznych i złożonych interfejsów użytkownika w sposób deklaratywny i modułowy
- struktura widoku i logika komponentu są trzymane blisko siebie z wykorzystaniem JavaScriptu lub TypeScriptu
- React buduje widoki w architekturze komponentowej, gdzie komponent to element oprogramowania posiadający dobrze wyspecyfikowany interfejs oraz zachowanie

Przykładowy kod:

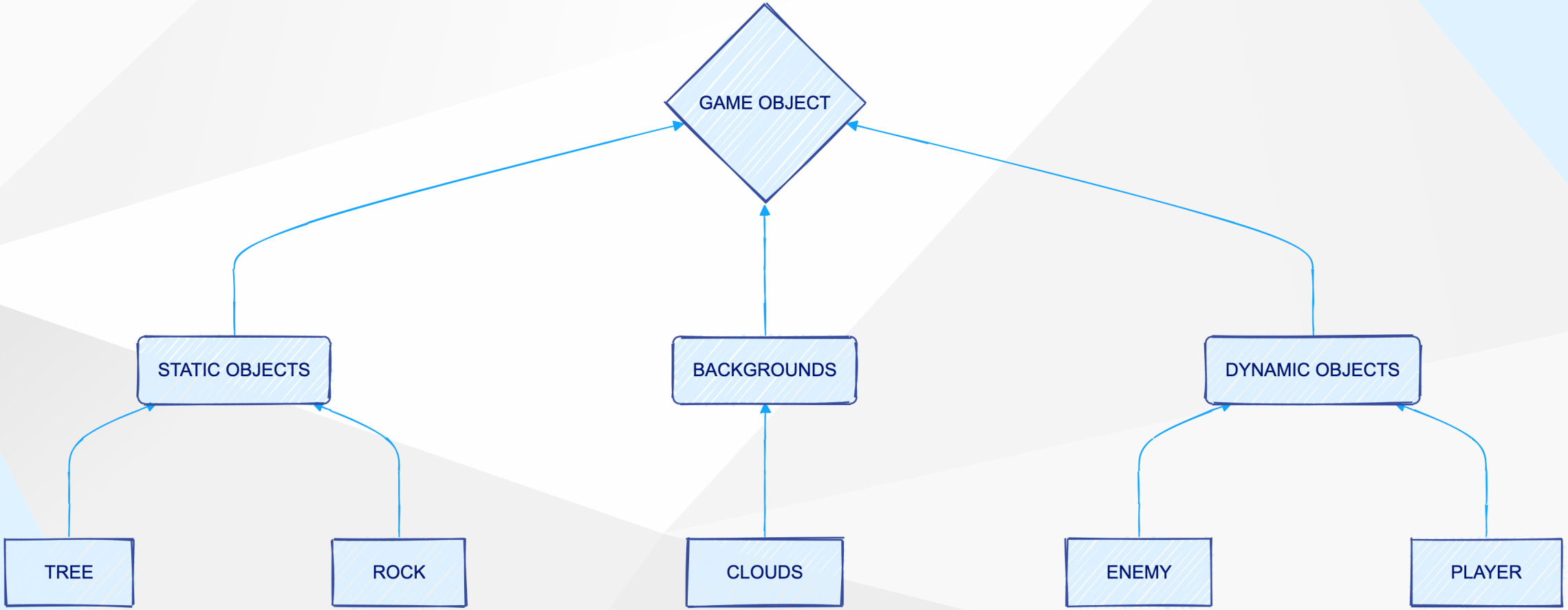
```tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';

import App from './App.tsx';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Z czego składa się React?

- `react` - podstawowa biblioteka obsługująca model kompozycyjny React'a
- `react-dom` - biblioteka udostępniająca specyficzne dla DOM metody, która dzieli się na: `react-dom/client` - moduł, który zezwala na tworzenie i używanie aplikacji React po stronie klienta oraz `react-dom/server` - moduł, który zezwala na budowanie komponentów React w wersji serwerowej

Architektura komponentowa

- komponenty mogą być wykorzystane w wielu aplikacjach
- skraca czas budowy nowych aplikacji i obniża koszty projektu - poprzez wykorzystanie istniejących komponentów
- wykorzystując istniejące komponenty po pierwsze, nie musimy ich pisać od nowa, po drugie testować i dokumentować
- poszczególne komponenty mogą być tworzone równolegle
- poprzez dobre wyspecyfikowanie interfejsów poszczególnych komponentów można łatwo zlecać implementacje komponentów na zewnątrz
- wspiera tzw. modyfikowalność aplikacji - konkretna funkcjonalność skupia się w dedykowanych komponentach programowych, więc w przypadku konieczności wprowadzenia zmiany, z reguły proces ten dotyczy modyfikacji jednego lub co najwyżej kilku komponentów a nie całej aplikacji
- spójność (ang. coherence) komponentów wspiera modyfikowalność - im większa spójność tym łatwiej modyfikować aplikację


GAME OBJECT
STATIC OBJECTS
BACKGROUNDS
DYNAMIC OBJECTS
TREE
ROCK
CLOUDS
ENEMY
PLAYER

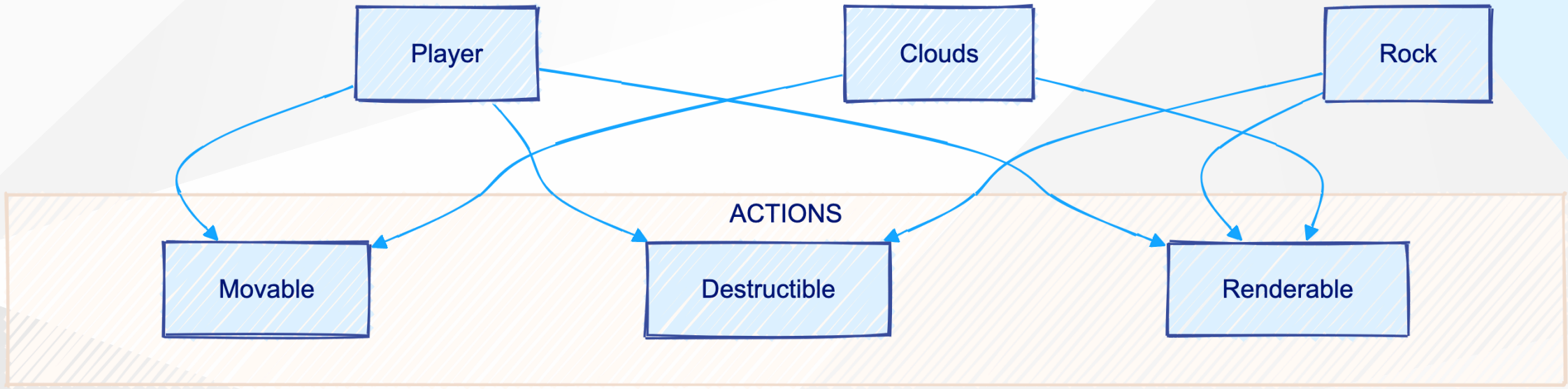
Loose Coupling

W idealnej architekturze komponenty powinny o sobie wiedzieć jak najmniej. Komponent `Player` nie powinien wiedzieć, jak zaimplementowany jest `Tree` . **Komunikacja odbywa się przez zdefiniowane kontrakty (interfejsy).**

Pozwala to na wymianę jednego komponentu na inny (np. zmianę `Tree` z wersji 2D na 3D) bez dotykania reszty logiki gry.

Kompozycja ponad dziedziczeniem

- współczesna architektura (np. ECS w grach `Entity-component-system` lub komponenty w Vue/Astro) promuje kompozycję
- zamiast tworzyć skomplikowane hierarchie klas, budujemy aplikację z małych, odizolowanych komponentów
- komponenty są łączone (komponowane) w większe jednostki funkcjonalne
- ułatwia to ponowne wykorzystanie kodu i testowanie
- zmiany w jednym komponencie mają minimalny wpływ na resztę systemu


Player
Clouds
Rock
ACTIONS
Movable
Destructible
Renderable

Kontrakt

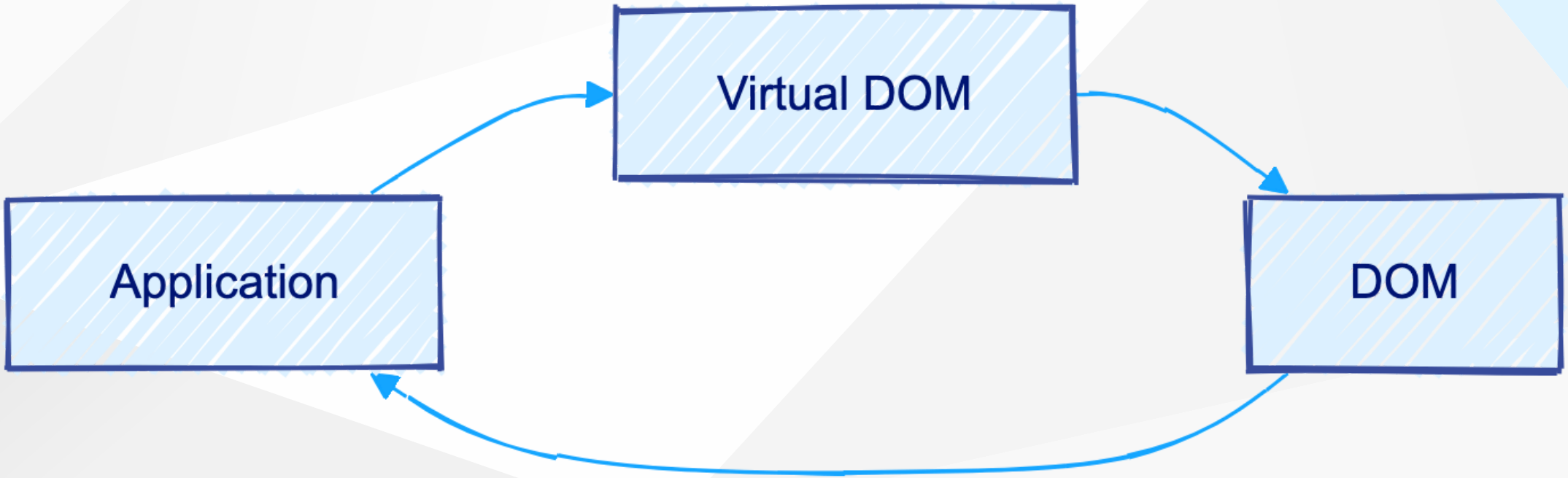
- każdy komponent powinien mieć jasno zdefiniowany interfejs (kontrakt)
- kontrakt określa, jakie dane komponent przyjmuje
- kontrakt powinien być jawny
- kontrakt powinien realizować zasadę Open/Closed (otwarty na rozszerzenia, zamknięty na modyfikacje) - komponent powinien być otwarty na rozbudowę (przez propsy/sloty), ale zamknięty na modyfikację (nie musimy zmieniać jego kodu źródłowego, by zmienić jego zachowanie)
- API komponentu powinno być stabilne i dobrze udokumentowane
- zmiany w API wymagają dyscypliny: `semver` , `deprecations` , `changelog`

Architektura komponentowa działa najlepiej, gdy komponenty mają wysoką spójność, niskie sprzężenie i stabilne kontrakty, a ich reużywalność jest wspierana przez wersjonowanie, testy kontraktowe i dokumentację.

React nie renderuje zmian bezpośrednio, ale poprzez Virtual-DOM

VDOM

- VirtualDOM to uproszczona reprezentacja struktury DOM (Document Object Model)
- React buduje drzewo Virtual DOM w sposób deklaratywny, a następnie przygotowuje na jego podstawie drzewo DOM w przeglądarce
- w przypadku wystąpienia zmian w strukturze, React na nowo przygotuje drzewo Virtual DOM dla zmienionych komponentów
- biblioteka ReactDOM porównuje aktualne i nowe drzewo Virtual DOM, a następnie aktualizuje tylko te miejsca, które wymagają zmiany
- VDOM należy traktować jako obiekt JS


Virtual DOM
Application
DOM

Zarządzanie poprzez DOM vs VDOM

	React (Virtual DOM)	Angular (Real DOM)	Svelte (Direct DOM)
Wydajność	Większa wydajność aktualizacji DOM	Aktualizacja DOM przy większych aplikacjach może być mniej wydajna	Bardzo wysoka, dzięki kompilacji
Złożoność	Średnia (deklaratywność + Virtual DOM)	Wysoka (dużo funkcji wbudowanych)	Niska (prostota + brak Virtual DOM)
Rozmiar pakietu	Większy (przez Virtual DOM runtime)	Duży	Najmniejszy
Ekosystem	Duży	Rozbudowany ekosystem	Najmniejszy

21.10.2026 19:17

Plusy użycia VDOM

- **efektywność aktualizacji**: React używa Virtual DOM do minimalizacji bezpośrednich operacji na rzeczywistym DOM, porównuje różnice (diff) między stanami wirtualnego DOM i aktualizuje tylko zmienione elementy w rzeczywistym DOM
- optymalizuje wydajność w aplikacjach z dużą ilością dynamicznych zmian
- **deklaratywność** - React pozwala programiście skupić się na co powinno się zmienić, a nie jak, to czyni kod bardziej zrozumiałym i mniej podatnym na błędy
- **cross-platform** - dzięki temu podejściu React może działać nie tylko w przeglądarce (React DOM), ale również na innych platformach (np. React Native)

Minusy użycia VDOM

- **warstwa abstrakcji** - dodatkowa warstwa (Virtual DOM) może prowadzić do narzutu wydajnościowego w prostych aplikacjach, gdzie bezpośrednia manipulacja DOM mogłaby być szybsza
- **brak pełnej kontroli nad DOM** - Virtual DOM ukrywa szczegóły rzeczywistego DOM, co może być problemem w specyficznych przypadkach wymagających szczegółowych optymalizacji

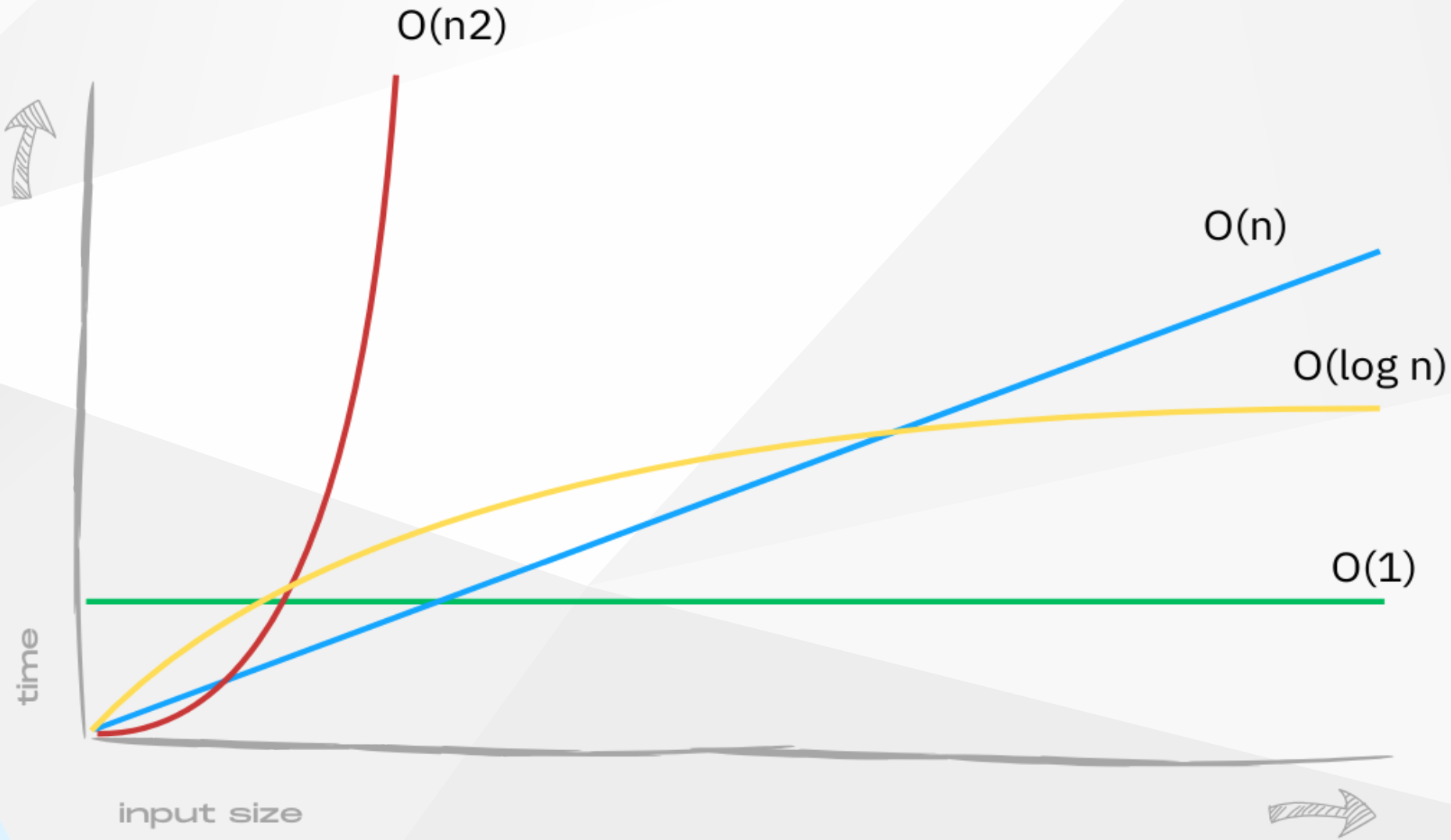
Virtual DOM

Zezwala na generowanie abstrakcyjnego DOM, który może być renderowany jako UI (User Interface) na wielu platformach:

- react-dom - przeglądarkowy DOM
- React Native - natywne aplikacje dla iOS i Android
- react-blessed - terminal
- react-canvas - elementy HTML Canvas
- react-vr / react 360 - aplikacje 3D

Diffing algorithm (algorytm różnicujący)

- pozwala Reactowi wykrywać zmiany i aktualizować tylko te fragmenty DOM, które naprawdę tego wymagają
- działa z złożonością (`Big O notation`) `O(n)` , gdzie n to liczba węzłów w drzewie, osiąga to dzięki rekursyjnemu porównywaniu drzew i użyciu kluczy w listach
- chociaż algorytm jest szybki, nadal istnieje pewien narzut związany z utrzymywaniem i porównywaniem Virtual DOM - jest to tzw. koszt abstracji
- od 2017 roku React używa algorytmu o nazwie Fiber, który wprowadza koncepcję przerwanych i asynchronicznych aktualizacji, co pozwala na bardziej responsywne aplikacje


O(n2)
O(n)
O(log n)
O(1)
time
input size

Concurrent Rendering

- pozwala Reactowi na bardziej elastyczne i responsywne aktualizacje interfejsu użytkownika
- umożliwia przerwanie długotrwałych operacji renderowania i kontynuowanie ich w późniejszym czasie
- wprowadza nowe mechanizmy, takie jak `startTransition`, które pozwalają na oznaczanie niepilnych aktualizacji

Przykładowy kod:

```
import { startTransition, useState } from 'react';
function App() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    startTransition(() => { setCount(c => c + 1) });
  };

  return (
    <div>
      <button onClick={handleClick}>Increment</button>
      <p>Count: {count}</p>
    </div>
  );
}
```

Elementy o różnych typach

- w procesie różnicowania React porównuje dwa drzewa Virtual DOM (aktualne i nowe)
- proces sprawdzania rozpoczyna od sprawdzenia typów elementów najwyższego poziomu (root)
- kiedy typy są różne (np. `SECTION` i `DIV`) - React niszczy stare drzewo, wraz z instancjami wcześniejszych komponentów i na podstawie nowego Virtual DOM renderuje na nowo aplikację

Podczas tworzenia drzewa na nowo:

- stare elementy DOM są niszczone
- instancje komponentów otrzymają informację, że komponent zostanie odmontowany
- nowe drzewo zostanie wyrenderowane w miejsce starego
- nowe komponenty otrzymają informację, że zostały zamontowane

Elementy DOM o tym samym typie

Kiedy elementy DOM mają ten sam typ:

- React sprawdza argumenty (właściwości) obu porównywanych elementów i w następnym kroku aktualizuje tylko te wartości, które wymagają zmiany, np. `styles`, `className`
- bez niszczenia drzewa idzie do dzieci i rozpoczyna różnicowanie na elementach potomnych

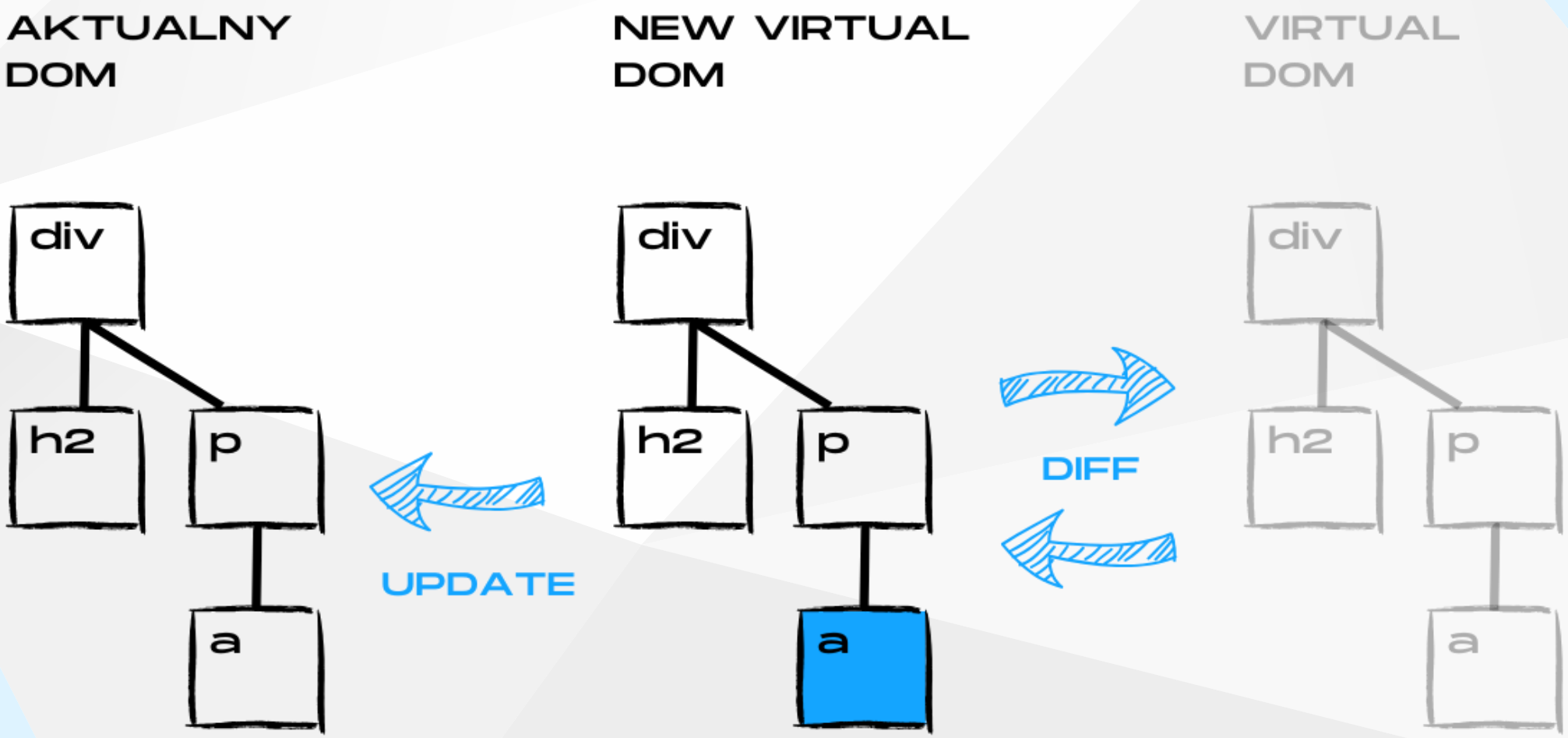
Komponenty elementów o tym samym typie

Kiedy są takie same:

- pomiędzy re-renderowaniami instancja komponentu pozostaje niezmieniona

Kiedy w komponencie wykryto zmianę:

- komponent zostanie poinformowany o potrzebie wykonania re-renderowania.


AKTUALNY DOM
NEW VIRTUAL DOM
VIRTUAL DOM
div
h2
p
a
UPDATE
DIFF

Rekurencja na dzieciach

- domyślnie React iteruje się poprzez listę dzieci w tym samym czasie i generuje mutacje w przypadku napotkania różnic
  Jeśli React znajdzie nowy element na końcu listy - doda go do niej
- gorzej jeśli nowy element pojawi się na początku lub w środku listy - React porównując elementy nie będzie w stanie wykryć, który element jest nowy - w związku z tym wyrenderuje całą listę od momentu wystąpienia różnicy
- tak przeprowadzona aktualizacja ma bardzo negatywny wpływ na wydajność aplikacji
- aby uniknąć takich sytuacji - elementy listy powinny otrzymać atrybut `key`

Key

- element kolekcji powinien mieć dodany atrybut `key`
- `key` powinien być unikalny i stabilny pomiędzy re-renderowaniami komponentu. Na jego podstawie React jest w stanie rozpoznać elementy, odpowiednio je porównać i podjąć decyzję czy powinny być ponownie wyrenderowane
- dzięki kluczom React jest w stanie optymalnie wyrenderować listę i pominąć zbędne re-renderowania komponentu, a zatem wykonać na drzewie DOM możliwie najmniej operacji (mutacji)
- jeśli `key` otrzyma nową wartość, React przerenderuje całą listę ponownie

Za aktualizację drzewa DOM odpowiada algorytm różnicujący (Diffing Algorithm). Proces aktualizacji drzewa DOM to rekoncyliacja.

Algorytm różnicujący jest tylko szczegółem implementacyjnym. Możemy go pominąć. Wówczas każda zmiana będzie powodowała re-render całej aplikacji.

Tworzenie elementów w aplikacji React

Przykładowy kod:

```
React.createElement(
  'div',
  {
    id: 'root_element',
    className: 'root-element-class',
    style: {
      borderTop: '1px solid black',
    }
  },
  "Hello!"
)
```

JSX

- składnia przypominająca HTML pozwalająca na używanie rozwiniętej składni HTML w kodzie JS
- składnia wzbogaca "HTML" o możliwość użycia zmiennych i wyrażeń

Przykładowy kod:

```
const Section = <section>
  <h1>title</h1>
  <h2 className="subtitle" id="hook_to_title">daily</h2>
  <ul>
    <li>ts</li>
    <li>react</li>
  </ul>
</section>;

createRoot(document.getElementById('root')!).render(Section);
```

Przykładowy kod:

```
const section = {
  title: 'title',
  subtitle: 'daily',
}

const Section = <section>
  <h1>{section.title}</h1>
  <h2 className="subtitle" id="hook_to_title">{section.subtitle}</h2>
  <ul>
    <li>ts</li>
    <li>react</li>
  </ul>
</section>;

createRoot(document.getElementById('root')!).render(Section);
```

Przykładowy kod:

```
const section = {
  title: 'title',
  subtitle: 'daily',
  items: [
    {
      id: 1,
      name: 'ts',
    },
    {
      id: 2,
      name: 'react',
    },
  ]
};

const Section = <section>
  <h1>{section.title}</h1>
  <h2 className="subtitle" id="hook_to_title">{section.subtitle}</h2>
  <ul>
    {section.items.map(item => <li key={item.id}>{item.name}</li>)}
  </ul>
</section>;
```

Komponenty w React

- komponent to podstawowy blok aplikacji Reactowej
- komponent to fragment interfejsu zarządzany przez React
- **funkcja komponentu** zawiera kod JavaScript/TypeScript, który kontroluje wygląd i zachowanie elementu
- komponent powinien być czysty: dla tych samych propsów i stanu zwraca ten sam JSX
- komponenty mogą być zagnieżdżane w dowolne struktury, podobnie jak HTML czy XML
- możemy przekazać do komponentu dowolne dane jako atrybuty, w taki sam sposób jak w HTMLu
- w przeciwieństwie do HTMLa poprzez atrybuty możemy przekazywać również obiekty (a nie tylko stringi).
- przekazane w ten sposób parametry nazywają się właściwościami komponentu (ang. Component properties, w skrócie `props`)

Przykładowy kod:

```
<MyComponent option={variable} title="text">
  Text or other components
</MyComponent>
```

Przykładowy kod:

```
// function component

const Greetings = ({ name }: { name: string }) => {

  return <h1>Hello, {name}</h1>
}
```

Komponenty funkcyjne były dawniej często nazywane bezstanowymi. Od wprowadzenia Hooków mogą zarządzać stanem i efektami, dlatego dziś traktujemy je jako podstawowy sposób pisania komponentów React.

Przykładowy kod:

```tsx
// function component in TS

interface GreetingsProps {
  name: string;
}

const Greetings = ({ name }: GreetingsProps) => {

  return <h1>Hello, {name}</h1>
}

const Greetings2: React.FC<GreetingsProps> = ({ name }) => {

  return <h1>Hello, {name}</h1>
}
```

Vite

Narzędzie do budowania aplikacji frontendowych. Wspiera wiele frameworków, w tym React. Vite jest szybszy niż CRA, szczególnie podczas uruchamiania i pracy w trybie deweloperskim.

Przykładowy kod:

```
npm create vite@latest my-app -- --template react-ts

# albo dla JavaScript
npm create vite@latest my-app -- --template react
```

Kiedy wybrać Vite?

- budujemy klasyczną aplikację SPA
- chcemy pełnej kontroli nad doborem bibliotek
- nie potrzebujemy SSR ani Server Components na starcie
- zależy nam na prostym setupie warsztatowym i szybkim feedback loop

Kiedy wybrać framework React?

Narzędzie	Kiedy ma sens
React Router framework	routing, loadery, akcje, bardziej kompletna architektura SPA
Next.js	SSR, Server Components, Server Actions, SEO, aplikacje full-stack
Remix	formularze, routing i praca z danymi w modelu bliskim web platform

Struktura projektu warsztatowego

Przykładowy kod:

```
src/
  api/          # komunikacja z backendem i klient HTTP
  components/   # komponenty wielokrotnego użycia
  hooks/        # custom hooks
  routes/       # widoki i konfiguracja routingu
  tests/        # konfiguracja testów i helpery
```


21.10.2026 19:17

Create React App (deprecated)

Wrapper dla aplikacji Reactowej. CRA ukrywa szczegóły implementacyjne, w szczególności konfigurację bundlera. Dziś traktujemy go jako rozwiązanie historyczne, spotykane głównie w starszych projektach.

Przykładowy kod:

```
npx create-react-app my-app --template typescript
```

Zadanie nr 1

20 minut

- sklonuj repozytorium szkoleniowe i uruchom przygotowany projekt React + TypeScript

Przykładowy kod:

```
git clone https://github.com/matwjablonski/react-open-10-26.git
cd react-open-10-26/my-books-app
npm ci
npm run dev
```

- zostaw aplikację uruchamianą przez `src/main.tsx` i główny komponent `src/App.tsx`
- zmodyfikuj `App.tsx`, tak aby wyświetlał informacje o aplikacji:

1. tytuł: "Witaj w naszej aplikacji książkowej"
2. polecane książki: listę `ul / li` a w niej 3 tytuły książek
3. stopkę z informacją: "Aplikacja przygotowana przez <imię i nazwisko>"

- przygotuj plik `src/data/books.ts` z tablicą minimum 3 książek, które wykorzystamy w kolejnych zadaniach
- każda książka powinna zawierać stabilne `id`, `title`, `author` oraz `publicationDate`

Stan i props

- `props` to dane przekazane do komponentu przez rodzica
- `state` to dane, które komponent przechowuje pomiędzy renderami
- render jest ponownie uruchamiany, gdy zmieni się stan komponentu lub dane przekazane przez rodzica
- props są tylko do odczytu - komponent nie powinien modyfikować danych otrzymanych od rodzica
- stan aktualizujemy przez funkcję aktualizującą zwróconą przez `useState`

Przykładowy kod:

```
type TodoProps = {
  title: string;
  completed?: boolean;
};

const Todo = ({ title, completed = false }: TodoProps) => (
  <article>
    <h3>{title}</h3>
    {completed && <p>Przeczytane</p>}
  </article>
);

const App = () => (
  <Todo title="Clean Code" />
);
```

State as snapshot

- stan w trakcie jednego renderu jest niezmiennym "zdjęciem" danych
- wywołanie funkcji aktualizującej planuje kolejny render, ale nie zmienia lokalnej zmiennej w już trwającym renderze
- gdy nowa wartość zależy od poprzedniej, używamy funkcji aktualizującej
- obiekty i tablice w stanie traktujemy niemutowalnie

Przykładowy kod:

```
const [count, setCount] = useState(0);

const increment = () => {
  setCount((current) => current + 1);
};
```

Wartość pochodna czy stan?

- jeśli wartość można policzyć z propsów albo istniejącego stanu, zwykle nie powinna być ona osobnym stanem
- stan powinien przechowywać minimalny zestaw danych potrzebny do odtworzenia widoku
- duplikowanie danych w stanie zwiększa ryzyko niespójności

Przykładowy kod:

```
const [books, setBooks] = useState<Book[]>([]);
const [query, setQuery] = useState('');

const filteredBooks = books.filter((book) =>
  book.title.toLowerCase().includes(query.toLowerCase())
);
```

Lifting state up

- gdy kilka komponentów potrzebuje tych samych danych, przenosimy stan do najbliższego wspólnego rodzica
- komponent rodzic przechowuje stan i przekazuje dane oraz callbacki przez props
- taki przepływ danych jest jawny i łatwy do debugowania

Przykładowy kod:

```
const BooksPage = () => {
  const [selectedBookId, setSelectedBookId] = useState<string | null>(null);

  return (
    <>
      <BooksList onSelectBook={setSelectedBookId} />
      <BookDetails bookId={selectedBookId} />
    </>
  );
};
```

Zadanie nr 2

20 minut

- wydziel tytuł strony do osobnego komponentu `Header`
- wydziel stopkę strony do komponentu `Footer`
- przygotuj komponenty `Books` oraz `Book`
- komponent `Books` powinien renderować książki z pliku `src/data/books.ts` przekazane przez `props`
- komponent `Book` powinien wyświetlić tytuł, autora oraz datę publikacji - dane te powinien otrzymać z `props`
- każda książka powinna mieć stabilne `id`, które wykorzystamy później w routingu i stanie aplikacji

21.10.2026 19:17

Aktualizacja stanu

- React musi wiedzieć, kiedy stan komponentu się zmienił
- nigdy nie zmieniaj stanu bezpośrednio!
- w komponentach funkcyjnych do zmiany stanu służy funkcja zwrócona przez `useState`
- aktualizacje stanu są kolejkowane i mogą zostać połączone w jeden render
- kiedy nowy stan zależy od poprzedniego, używamy funkcji aktualizującej
- obiekty i tablice aktualizujemy przez tworzenie nowych referencji

Kolejkowanie aktualizacji

Przykładowy kod:

```
const [count, setCount] = useState(0);

const handleClick = () => {
  setCount((value) => value + 1);
  setCount((value) => value + 1);
  setCount((value) => value + 1);
};
```

Niemutowalne aktualizacje

Przykładowy kod:

```
type Book = {
  id: string;
  title: string;
  read: boolean;
};

const [books, setBooks] = useState<Book[]>([]);

const markAsRead = (id: string) => {
  setBooks((currentBooks) =>
    currentBooks.map((book) =>
      book.id === id ? { ...book, read: true } : book
    )
  );
};
```

Resetowanie stanu przez key

- React zachowuje stan komponentu, dopóki komponent pozostaje w tym samym miejscu drzewa
- zmiana `key` informuje Reacta, że to inna instancja komponentu
- przydaje się to np. przy resetowaniu formularza po zmianie edytowanego rekordu

Przykładowy kod:

```
<BookForm key={selectedBookId} bookId={selectedBookId} />
```

Zadanie nr 3

10 minut

- w komponencie `App` dodaj stan przechowujący identyfikatory przeczytanych książek
- do komponentu `Book` dodaj przycisk, który będzie miał dwa stany: "Dodaj do przeczytanych" lub "Usuń z przeczytanych"
- po kliknięciu w przycisk stan powinien się zaktualizować, a obok tytułu książki powinna pojawić się informacja, że książka została przeczytana
- do wykonania zadania wykorzystaj `useState`, `props` i `onClick`

Pytania po zadaniu

PYTANIE

Który komponent powinien być właścicielem tego stanu i dlaczego?

PYTANIE

Która tożsamość książki musi pozostać stabilna po zmianie kolejności lub usunięciu?

PYTANIE

Jak sprawdzić, że aktualizacja jednej książki nie zmienia pozostałych?

Kontrakt komponentu

- komponent powinien mieć jawny kontrakt danych wejściowych
- w projekcie TypeScript opisujemy go przez `type` albo `interface`
- kontrakt powinien być możliwie mały i stabilny
- wartości domyślne w komponentach funkcyjnych zapisujemy w parametrach

Przykładowy kod:

```tsx
type BookProps = {
  title: string;
  author: string;
  read?: boolean;
  onToggleRead: () => void;
};

const Book = ({ title, author, read = false, onToggleRead }: BookProps) => (
  <article>
    <h3>{title}</h3>
    <p>{author}</p>
    <button onClick={onToggleRead}>
      {read ? 'Usuń z przeczytanych' : 'Dodaj do przeczytanych'}
    </button>
  </article>
);
```

Props jako API komponentu

- props powinny opisywać intencję, a nie szczegóły implementacji
- unikaj przekazywania dużych obiektów, jeśli komponent potrzebuje tylko kilku pól
- callbacki nazywaj od zdarzenia lub intencji: `onSubmit`, `onSelectBook`, `onToggleRead`
- kiedy komponent ma tylko układać inne elementy, użyj kompozycji i `children`

Legacy: PropTypes

- `PropTypes` można spotkać w starszych projektach bez TypeScriptu
- w nowym kodzie warsztatowym używamy TypeScriptu
- jeśli utrzymujemy stary projekt JS, `PropTypes` nadal mogą pomóc wykryć błędne propsy w development

Zadanie nr 4

10 minut

- za pomocą TypeScript opisz propsy wchodzące do komponentów `Book` oraz `Books`
- przygotuj typ `BookItem`, który opisze dane jednej książki
- komponent `Books` powinien przyjmować tablicę `BookItem[]`
- komponent `Book` powinien przyjmować dane jednej książki, informację `isRead` oraz callback do zmiany stanu przeczytania
- wyeksportuj typy z jednego miejsca, np. `src/types/book.ts`

Zdarzenia

- zdarzenia w React rejestrujemy bezpośrednio na elementach przez propsy zaczynające się od `on`
- funkcja obsługi zdarzenia jest częścią logiki komponentu
- obsługa zdarzeń często aktualizuje stan albo wywołuje callback przekazany przez rodzica

Przykładowe propsy dla zdarzeń:

- `onClick` - zdarzenie kliknięcia w element
- `onChange` - zdarzenie zmiany wartości pola formularza
- `onSubmit` - zdarzenie wysłania formularza

Synthetic Event

- obiekty zdarzeń w React mają typy opakowujące natywne zdarzenia przeglądarki
- interfejs zdarzenia jest zgodny z natywnym zdarzeniem, ale React normalizuje zachowanie między przeglądarkami
- w TypeScript używamy typów dopasowanych do elementu, np. `React.ChangeEvent<HTMLInputElement>`
- aby obsłużyć fazę przechwytywania, dodajemy suffix `Capture`, np. `onClickCapture`

Przykładowy kod:

```
const Input = () => {
  const [value, setValue] = useState('');

  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
    setValue(event.currentTarget.value);
  };

  return <input value={value} onChange={handleChange} />;
};
```

Przykładowy kod:

```tsx
type SaveButtonProps = {
  onSave: () => void;
};

const SaveButton = ({ onSave }: SaveButtonProps) => (
  <button type="button" onClick={onSave}>
    Zapisz
  </button>
);
```

Formularze

- domyślnie pola formularza w React oznaczone są jako pole niekontrolowane (uncontrolled components)
- pola niekontrolowane nie są kontrolowane w pełni przez mechanizm Reacta (za zmianę ich wartości odpowiada przeglądarka)
- jeśli do pola formularza dodany atrybut `value`, wówczas pole zmienia się w pole kontrolowane (controlled component)
- pole kontrolowane jest kontrolowane przez Reacta a zatem zmiana jego wartości może odbyć się tylko za pośrednictwem mechanizmu Reacta (tj. poprzez zmianę stanu lub propsów - w większości przypadków poprzez zmianę stanu)
- kiedy pole jest kontrolowane tylko komponent może zmienić jego wartość (na UI pole nie będzie reagowało bez implementacji funkcji zmiany wartości)
- kod `<input value={this.state.myValue} />` sprawia, że pole staje się niemożliwe do edycji ręcznie
- elementy typu `<select>` i `<textarea>` również mogą przyjmować atrybut `value`

Zarządzanie formularzem za pomocą stanu

- do zarządzania formularzem w React możemy wykorzystać stan komponentu
- każda zmiana wartości pola formularza powinna być obsłużona przez funkcję, która zaktualizuje stan komponentu

Przykładowy kod:

```
import React, { useState } from 'react';

const MyForm = () => {
  const [inputValue, setInputValue] = useState('');

  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
    setInputValue(event.target.value);
  };

  return (
    <form>
      <input type="text" value={inputValue} onChange={handleChange} />
    </form>
  );
};
```

Zarządzanie formularzem za pomocą referencji

- do zarządzania formularzem w React możemy tez wykorzystać referencje
- referencje pozwalają na bezpośredni dostęp do elementu DOM

Przykładowy kod:

```tsx
import React, { useRef } from 'react';

const MyForm = () => {
  const inputRef = useRef<HTMLInputElement>(null);
  const handleSubmit = (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    inputRef.current && console.log(inputRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" ref={inputRef} />
      <button type="submit">Submit</button>
    </form>
  );
};
```

Kiedy używać stanu a kiedy referencji?

- stan jest bardziej odpowiedni, gdy wartość pola formularza jest potrzebna do renderowania komponentu lub do walidacji danych
- referencje są bardziej odpowiednie, gdy potrzebujemy bezpośredniego dostępu do elementu DOM, np. do ustawienia focus lub do integracji z bibliotekami zewnętrznymi
- w większości przypadków zarządzanie formularzem za pomocą stanu jest bardziej zgodne z filozofią Reacta i ułatwia utrzymanie spójności
- JEDNAK w niektórych sytuacjach użycie referencji może być bardziej efektywne, np. gdy mamy bardzo duży formularz i chcemy uniknąć nadmiernych renderów lub gdy chcemy szybko odczytać wartości pól dopiero przy submit
- wybór między stanem a referencjami zależy od konkretnego przypadku użycia

Zadanie nr 5

30 minut

- przygotuj komponenty `Input` oraz `Form`
- komponent `Input` powinien przyjmować wszystkie standardowe atrybuty elementu `input` oraz prop `label`
- komponent `Form` powinien przyjmować `onSubmit` oraz `children`
- na podstawie tych komponentów przygotuj `ContactForm`, który umieścisz w stopce
- wysłanie formularza ma na razie wypisać dane w konsoli przeglądarki
- formularz powinien zawierać pola `name`, `message`, `phoneNumber`

Zagnieżdżanie komponentów

- komponenty mogą być zagnieżdżane
- każdy kod JSX, który jest przekazany pomiędzy znacznikiem otwarcia i zamknięcia komponentu, będzie dostępny w tym komponencie jako props `children`
- jako dzieci komponentu możemy zagnieżdżać stringi, tablice elementów JSX i wszystko co wypełnia typ `ReactNode`

Przykładowy kod:

```
const LastItem = () => <p>I am last!</p>

const List = ({ data, children }) => {
  return (
    <ul>
      {data.map(item => <li key={item.id}>{item.title}</li>)}
      {children}
    </ul>
  )
}

const App = () => (
  <List data={[]}>
    <LastItem />
  </List>
)

export default List;
```

ReactNode

`ReactNode` to szeroko stosowany typ w ekosystemie React, który może reprezentować dowolne treści renderowalne w aplikacji. Jest używany między innymi do określenia, co może być przekazane jako dzieci (`children`) lub zawartość komponentu.

Poniżej znajduje się uproszczony model typu. Pełna definicja zależy od używanej wersji typów Reacta.

Przykładowy kod:

```
type ReactNode =
  | ReactElement
  | string
  | number
  | bigint
  | boolean
  | null
  | undefined
  | Iterable<ReactNode>
  | ReactPortal
  | Promise<AwaitedReactNode>;
```

ReactNode czy ReactElement?

	ReactNode	ReactElement
Opis	Dowolna treść, która może być wyrenderowana.	Pojedynczy element React (wynik React.createElement).
Może zawierać	String, number, JSX, null, array, boolean, etc.	Tylko elementy React (JSX lub React.createElement).
Typowy użytek	children props, zawartość renderowalna.	Konkretne elementy React.

Prop drilling (także: threading) to proces przekazywania danych przez propsy w dół drzewa komponentów do komponentu docelowego.

Przykładowy kod:

```
const UserBox = ({ user }) => <div>{user.name}</div>;

const UserProfile = ({ user }) => <UserBox user={user} />;

const Menu = ({ user }) => <UserProfile user={user} />;

const Nav = ({ user }) => <Menu user={user} />;

const App = () => {
  const user = {
    name: 'Mateusz',
  };

  return <Nav user={user} />
}
```

Zadanie nr 6

10 minut

- załóż, że nagłówek `Header` może przyjąć prop `children`
- przygotuj komponent `UserInfo`, który wyświetli informację o zalogowanym użytkowniku
- przekaż `UserInfo` do `Header` przez `children`

Cykl życia renderowania

- współczesny React opisujemy przez fazy renderowania, commit i efekty
- funkcja komponentu powinna być czysta: z tych samych propsów i stanu zwraca ten sam JSX
- efekty uboczne wykonujemy poza renderem, zwykle w `useEffect`
- cleanup efektu odpowiada za odpięcie subskrypcji, timerów i listenerów

Render, commit, effects

Faza	Co się dzieje	Czego unikać
Render	React wywołuje komponenty i buduje opis UI	efektów ubocznych
Commit	React aktualizuje DOM	kosztownych obliczeń blokujących UI
Effects	React uruchamia efekty po aktualizacji widoku	braku cleanupu

Legacy: metody cyklu życia klas

- `componentDidMount`, `componentDidUpdate` i `componentWillUnmount` spotkamy w starszych komponentach klasowych
- metody `UNSAFE_componentWillMount`, `UNSAFE_componentWillReceiveProps` i `UNSAFE_componentWillUpdate` traktujemy jako sygnał długu technologicznego
- w nowym kodzie warsztatowym używamy komponentów funkcyjnych i hooków

Error Boundary

- Error Boundary przechwytuje błędy renderowania w poddrzewie komponentów
- pozwala pokazać fallback zamiast całkowicie zepsutego widoku
- nadal implementuje się go jako komponent klasowy albo wykorzystuje gotowe biblioteki/frameworki

Przykładowy kod:

```
class ErrorBoundary extends React.Component<Props, State> {
  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: Error) {
    reportError(error);
  }
}
```

Nie jest zalecane ręczne manipulowanie strukturami DOM wygenerowanymi przez Reacta. Gdy potrzebujemy dostępu do elementu DOM, używamy referencji.

Przykładowy kod:

```tsx
const Form = () => {
  const inputRef = useRef<HTMLInputElement>(null);

  return (
    <form>
      <input ref={inputRef} />
      <button type="button" onClick={() => inputRef.current?.focus()}>
        Ustaw focus
      </button>
    </form>
  );
};
```

Przykładowy kod:

```
const previousValueRef = useRef<string | null>(null);

const handleChange = (value: string) => {
  previousValueRef.current = value;
};
```

Manipulacje na DOM

Aby uzyskać dostęp do elementu DOM możemy użyć hooka `useRef`.

Ponadto:

- wartości referencji będą dostępne dopiero po wyrenderowaniu elementu
- referencje wykorzystujemy głównie do focusu, pomiarów DOM i integracji z bibliotekami zewnętrznymi
- przechowywanie danych w referencji nie powoduje re-renderowania komponentu po zmianie tych danych

Zadanie nr 7

15 minut

- sprawdź wysokość listy książek (użyj `useRef` oraz `getBoundingClientRect`)
- jeśli wysokość jest wyższa niż 500px to wypisz nad listą informację: "Masz masę książek"
- jeśli jest mniejsza to wypisz "Zbieraj dalej"
- wykonaj pomiar po wyrenderowaniu listy, a nie w trakcie renderowania komponentu

Ref jako prop

W React 19 komponent może przyjąć `ref` jako zwykły prop i przekazać go dalej do elementu DOM.

Przykładowy kod:

```tsx
type InputProps = React.ComponentProps<'input'>;

const Input = ({ ref, ...props }: InputProps) => (
  <input {...props} ref={ref} />
);
```


21.10.2026 19:17

Przykładowy kod:

```
function Parent() {
  const inputRef = useRef<HTMLInputElement>(null);

  return (
    <div>
      <Input ref={inputRef} />
      <button onClick={() => inputRef.current?.focus()}>
        Skup się na polu tekstowym
      </button>
    </div>
  );
}
```

Legacy: forwardRef

- w starszych wersjach Reacta do przekazywania `ref` przez komponent funkcyjny używano `forwardRef`
- w React 19 `forwardRef` jest potrzebny głównie przy utrzymaniu starszego kodu

Zadanie nr 8

20 minut

- do napisanego wcześniej formularza kontaktowego dodaj przycisk "Napisz do nas", który po kliknięciu "postawi" kursor w pierwszym polu formularza
- użyj do wykonania tego zadania `useRef` i `.focus()`
- użyj `useRef` w komponencie rodzica i przekaż `ref` do komponentu pola formularza jako zwykły prop `ref`

React Hooks

- hooki pozwalają komponentom funkcyjnym korzystać ze stanu, efektów, referencji, kontekstu i mechanizmów współbieżnego renderowania
- hook jest funkcją wywoływaną w komponencie React albo w innym hooku
- nazwa własnego hooka powinna zaczynać się od `use`
- hooki opierają się na kolejności wywołań w trakcie renderowania komponentu

Kolejność wywoływania Hooków jest ważna!

Reguły hooków

- wywołuj hooki tylko na najwyższym poziomie komponentu lub własnego hooka
- nie wywołuj hooków w pętlach, warunkach, callbackach ani po `return`
- hooki muszą być wywoływane w tej samej kolejności przy każdym renderze
- reguły hooków powinny być pilnowane przez `eslint-plugin-react-hooks`
- w uproszczeniu: podczas renderowania React kojarzy kolejne wywołania hooków z wewnętrznymi slotami danej instancji komponentu
- dlatego zmiana kolejności wywołań hooków oznacza, że stan, efekt lub ref może zostać przypisany do niewłaściwego miejsca
- `use()` jest wyjątkiem: można wywoływać go warunkowo i w pętli, ale nie można opakować go w `try/catch`

Przykładowy kod:

```
const useBookSearch = (books: Book[]) => {
  const [query, setQuery] = useState('');

  const results = books.filter((book) =>
    book.title.toLowerCase().includes(query.toLowerCase())
  );

  return { query, setQuery, results };
};
```

useState

- służy do lokalnego stanu komponentu
- zwraca krotkę: bieżącą wartość i funkcję aktualizującą
- inicjalizator może być funkcją, jeśli obliczenie wartości początkowej jest kosztowne
- kiedy kolejny stan zależy od poprzedniego, używamy funkcji aktualizującej
- osobne wartości stanu grupujemy według domeny, a nie według typu pola

Przykładowy kod:

```tsx
const VoteButton = ({ initialVotes }: { initialVotes: number }) => {
  const [votes, setVotes] = useState(() => initialVotes);

  const vote = () => {
    setVotes((currentVotes) => currentVotes + 1);
  };

  return (
    <button type="button" onClick={vote}>
      Głosuj ({votes})
    </button>
  );
};
```

useReducer

- przydaje się, gdy stan ma kilka możliwych akcji i reguł przejścia
- dobrze sprawdza się przy formularzach, filtrach, kreatorach i złożonych listach
- reducer powinien być czystą funkcją
- akcje warto modelować jako union type w TypeScript

Przykładowy kod:

```
type BooksAction =
  | { type: 'book/read'; id: string }
  | { type: 'book/remove'; id: string };

const booksReducer = (state: Book[], action: BooksAction) => {
  switch (action.type) {
    case 'book/read':
      return state.map((book) =>
        book.id === action.id ? { ...book, read: true } : book
      );
    case 'book/remove':
      return state.filter((book) => book.id !== action.id);
  }
};
```

Przykładowy kod:

```
const BooksList = ({ initialBooks }: { initialBooks: Book[] }) => {
  const [books, dispatch] = useReducer(booksReducer, initialBooks);

  return books.map((book) => (
    <Book
      key={book.id}
      book={book}
      onRead={() => dispatch({ type: 'book/read', id: book.id })}
      onRemove={() => dispatch({ type: 'book/remove', id: book.id })}
    />
  ));
};
```

Zadanie nr 9

15 minut

- do każdej książki dodaj przycisk "Głosuj na tę pozycję"
- na liście pokaż informację, ile głosów oddano na daną pozycję
- użyj `useState` z inicjalizatorem funkcyjnym
- wartość początkowa powinna wynikać z operacji `<liczba liter w nazwisku autora> * 2 / 5`
- pamiętaj, że liczba głosów musi być liczbą całkowitą, użyj `Math.round()` albo `Math.floor()`

useEffect

- `useEffect` synchronizuje komponent z systemem zewnętrznym
- efektem może być subskrypcja, timer, listener, integracja z API przeglądarki albo ręcznie zarządzana biblioteka zewnętrzna
- efekt nie powinien służyć do obliczania wartości, które można policzyć podczas renderowania
- zanim dodasz efekt, zapytaj: z czym zewnętrznym synchronizuje się komponent?

Kiedy nie potrzebujesz useEffect?

- do obliczeń pochodnych z propsów lub stanu
- do obsługi zdarzeń użytkownika, np. kliknięcia lub submitu
- do resetowania stanu, jeśli wystarczy `key`
- do synchronizacji dwóch stanów, jeśli jeden można policzyć z drugiego

Przykładowy kod:

```
const WindowTitleHandler = () => {
  const [booksCount, setBooksCount] = useState(0);

  useEffect(() => {
    document.title = `Twój zbiór książek liczy ${booksCount} książek`;
  }, [booksCount]);

  return <button onClick={() => setBooksCount((count) => count + 1)}>
    Dodaj książkę
  </button>;
};
```

W StrictMode React w development może uruchomić efekt i cleanup dodatkowy raz, aby wykryć niepoprawne założenia w kodzie efektu.

Przykładowy kod:

```
const WindowResizeWatcher = () => {
  const [windowWidth, setWindowWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResizeAction = () => {
      setWindowWidth(window.innerWidth);
    };

    window.addEventListener('resize', handleResizeAction);

    return () => {
      window.removeEventListener('resize', handleResizeAction);
    };
  }, []);

  return <div>
    Moje okno ma teraz {windowWidth}px szerokości
  </div>;
};
```

Tablica zależności

- zależności opisują wartości reaktywne używane przez efekt
- pustej tablicy używamy tylko wtedy, gdy efekt nie zależy od propsów ani stanu
- brak tablicy oznacza uruchomienie po każdym renderze
- reguła `exhaustive-deps` zwykle ma rację - lepiej zmienić kod niż wyciszać ostrzeżenie

Przykładowy kod:

```
useEffect(() => {
  document.title = title;
}, [title]);

useEffect(() => {
  const connection = connect(roomId);
  return () => connection.disconnect();
}, [roomId]);
```

Race condition i cleanup

Przykładowy kod:

```
useEffect(() => {
  const controller = new AbortController();

  fetch(`/api/books?query=${query}`, { signal: controller.signal })
    .then((response) => response.json())
    .then(setBooks)
    .catch((error) => {
      if (error.name !== 'AbortError') setError(error);
    });

  return () => controller.abort();
}, [query]);
```

useEffectEvent

- pozwala wydzielić nie-reaktywną logikę używaną wewnątrz efektu
- przydaje się, gdy efekt powinien reagować na jedną zależność, ale potrzebuje aktualnej wartości innej zmiennej
- zmniejsza pokusę wyciszania `exhaustive-deps`
- Effect Event wywołujemy wyłącznie z efektu w tym samym komponencie lub hooku
- aktualny `eslint-plugin-react-hooks` rozpoznaje Effect Events i pomija je w tablicy zależności

Przykładowy kod:

```
import { useEffect, useEffectEvent } from 'react';

const onConnected = useEffectEvent(() => {
  showNotification('Połączono', theme);
});

useEffect(() => {
  const connection = createConnection(roomId);
  connection.on('connected', onConnected);
  connection.connect();

  return () => connection.disconnect();
}, [roomId]);
```

useRef

- przechowuje mutowalną wartość pomiędzy renderami
- zmiana `ref.current` nie powoduje re-renderowania
- najczęstsze zastosowania: focus, pomiary DOM, id timeoutu, integracje z zewnętrznymi bibliotekami
- nie używaj `useRef` do danych, które mają wpływać na widok

Przykładowy kod:

```tsx
const inputRef = useRef<HTMLInputElement>(null);

return (
  <button type="button" onClick={() => inputRef.current?.focus()}>
    Ustaw focus
  </button>
);
```

Zadanie nr 10

10 minut

- zmień tytuł okna przeglądarki (`document.title`) tak, aby zawierał liczbę książek dostępnych w naszej aplikacji
- tytuł powinien brzmieć: "Twój zbiór książek liczy książek"
- wykorzystaj `useEffect`
- zadbaj o poprawną tablicę zależności i sprawdź zachowanie w `StrictMode`
- po odmontowaniu komponentu przywróć domyślny tytuł strony

Pytania po zadaniu

PYTANIE

Z jakim zewnętrznym systemem synchronizuje się ten efekt?

PYTANIE

Które zachowanie w StrictMode ujawni brak idempotencji lub poprawnego cleanupu?

PYTANIE

Kto powinien przywrócić poprzedni stan, gdy kilka widoków korzysta z tego samego zasobu?

useLayoutEffect

- zachowanie podobne do `useEffect`
- wywołuje się synchronicznie po zmianach DOM, ale przed odświeżeniem ekranu
- przydatny przy pomiarach layoutu, które muszą wpłynąć na render przed pokazaniem widoku użytkownikowi
- używaj rzadko, bo może blokować malowanie

21.10.2026 19:17

useDeferredValue

- pozwala odroczyć aktualizację wartości, która nie musi być natychmiastowa
- jest dostępny od React 18 - nie jest nowym hookiem React 19
- przydaje się przy filtrowaniu dużych list lub kosztownym renderowaniu wyników
- interfejs może reagować na wpisywanie szybko, a lista wyników odświeży się z niższym priorytetem
- nie ogranicza liczby zapytań sieciowych - do tego nadal potrzebny jest debounce albo cache

Przykładowy kod:

```
const [query, setQuery] = useState('');
const deferredQuery = useDeferredValue(query);

const results = filterBooks(books, deferredQuery);
```

useTransition czy useDeferredValue?

Potrzeba	API
kontrolujemy wywołanie aktualizacji stanu	`useTransition`
otrzymujemy wartość i chcemy odroczyć jej użycie	`useDeferredValue`
chcemy ograniczyć liczbę zapytań	debounce / cache

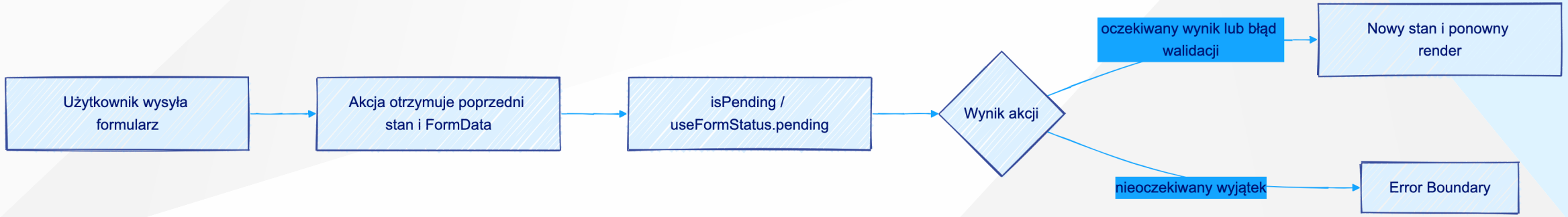
Oba hooki zmieniają priorytet renderowania, ale nie sprawiają, że kosztowna operacja sama staje się szybsza.

useMemo i useCallback

- `useMemo` zapamiętuje wynik obliczenia pomiędzy renderami
- `useCallback` zapamiętuje referencję funkcji pomiędzy renderami
- używaj ich po pomiarze albo gdy stabilna referencja jest częścią kontraktu komponentu
- nie traktuj memoizacji jako domyślnego sposobu pisania każdego komponentu

useActionState

- pozwala powiązać akcję formularza ze stanem wyniku tej akcji
- dobrze pasuje do submitu formularza, walidacji i komunikatów błędów
- zwraca bieżący stan, akcję formularza i informację `isPending`
- akcja może być asynchroniczna i otrzymuje poprzedni stan jako pierwszy argument


Użytkownik wysyła formularz
Akcja otrzymuje poprzedni stan i FormData
isPending / useFormStatus.pending
Wynik akcji
oczekiwany wynik lub błąd walidacji
Nowy stan i ponowny render
nieoczekiwany wyjątek
Error Boundary

Przykładowy kod:

```
const [state, submitBook, isPending] = useActionState(
  async (_state: FormState, formData: FormData) => {
    return saveBook(formData);
  },
  { error: null }
);

return <form action={submitBook}>...</form>;
```

useFormStatus

- hook z `react-dom`
- udostępnia `pending`, `data`, `method` i `action` najbliższego formularza nadrzędnego
- przydaje się w reużywalnych przyciskach submit i komunikatach o wysyłaniu
- nie odczyta statusu formularza wyrenderowanego w tym samym komponencie ani w komponencie potomnym

Przykładowy kod:

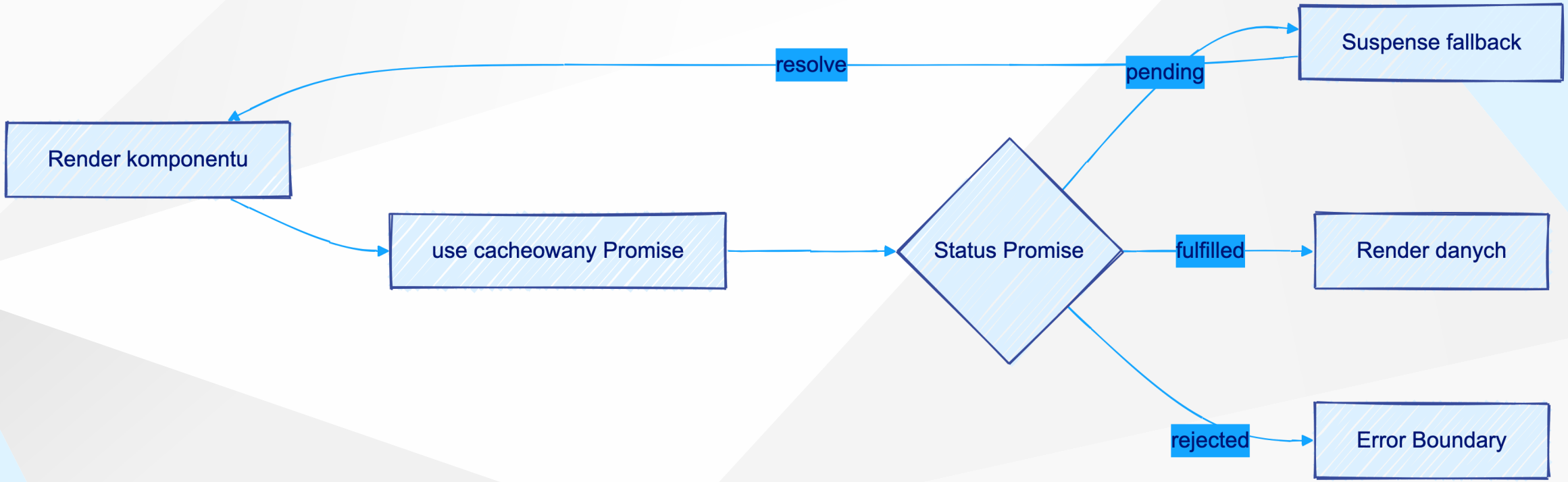
```
import { useFormStatus } from 'react-dom';

const SubmitButton = () => {
  const { pending } = useFormStatus();

  return <button disabled={pending}>Zapisz</button>;
};
```

use()

- pozwala odczytać wartość Promise albo kontekstu podczas renderowania
- przy odczycie Promise współpracuje z Suspense
- odrzucony Promise obsługuje najbliższy Error Boundary
- mimo nazwy nie jest hookiem - może być wywołany warunkowo albo w pętli

Przykładowy kod:

```
const BookDetails = ({ bookPromise }: Props) => {
  const book = use(bookPromise);

  return <h2>{book.title}</h2>;
};
```


Render komponentu
resolve
pending
Suspense fallback
use cacheowany Promise
Status Promise
fulfilled
Render danych
rejected
Error Boundary

use() – granice odpowiedzialności

- Promise musi być cache'owany i zachowywać tę samą referencję między renderami
- nie twórz `fetch()` bezpośrednio w renderze - każdy render utworzy nowy Promise
- błędów z `use(promise)` nie przechwytujemy przez `try/catch` w komponencie
- `use(context)` może być warunkowe i odczytuje najbliższy provider albo wartość domyślną
- w aplikacjach frameworkowych preferuj mechanizm pobierania danych oferowany przez framework

Zadanie nr 11

15 minut

- dodaj funkcjonalność usuwania książek
- obok przycisku przeczytane dodaj przycisk "Usuń"
- po kliknięciu książka powinna zniknąć z listy a lista się odświeżyć
- przebuduj zarządzanie listą książek z `useState` na `useReducer`
- przygotuj akcje dla oznaczania książki jako przeczytanej, cofania oznaczenia i usuwania książki
- zamodeluj akcje jako union type w TypeScript

Komunikacja z API

- React nie dostarcza jednej wbudowanej warstwy komunikacji z backendem
- w prostej aplikacji możemy użyć `fetch` i własnego hooka
- w aplikacji produkcyjnej zwykle oddzielamy server state od lokalnego stanu UI
- dla server state domyślnym wyborem w tym szkoleniu będzie TanStack Query

Server state vs client state

Rodzaj stanu	Przykłady	Typowe narzędzie
Client state	otwarty modal, filtr, theme, krok formularza	`useState`, `useReducer`, Context
Server state	czytelnicy, książki z API, status zapytania, cache	TanStack Query
Routing state	parametry URL, aktywna ścieżka, search params	React Router

useEffect + fetch jako punkt wyjścia

- wymaga ręcznej obsługi `isLoading`, `error`, anulowania zapytań i wyścigu zapytań
- nie rozwiązuje cache, retry, deduplikacji i synchronizacji danych między komponentami

21.10.2026 19:17

Przykładowy kod:

```tsx
const Readers = () => {
  const [readers, setReaders] = useState<Reader[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    const controller = new AbortController();
    const fetchReaders = async () => {
      setIsLoading(true);
      setError(null);
      try {
        const res = await fetch('/users', { signal: controller.signal });
        if (!res.ok) throw new Error('Nie udało się pobrać czytelników');
        const data = await res.json();

        if (!controller.signal.aborted) setReaders(data);
      } catch (err) {
        if (!controller.signal.aborted) setError(err as Error);
      } finally {
        if (!controller.signal.aborted) setIsLoading(false);
      }
    };

    fetchReaders();
    return () => controller.abort();
  }, []);

  if (isLoading) return <p>Ładowanie...</p>;
  if (error) return <p>Nie udało się pobrać czytelników.</p>;
  return <ReadersList readers={readers} />;
};
```

Własny hook do API

- ukrywa szczegóły techniczne pobierania danych
- ułatwia ponowne użycie logiki w kilku komponentach
- nadal wymaga samodzielnej obsługi cache, retry i synchronizacji

Przykładowy kod:

```
const useReaders = () => {
  const [state, setState] = useState<ReadersState>({
    data: [],
    isLoading: true,
    error: null,
  });

  // fetch + AbortController + setState
  return state;
};
```

Zadanie nr 12

25 minut

- endpoint `https://jsonplaceholder.typicode.com/users` zwraca listę użytkowników (czytelnicy)
- przygotuj komponent `Readers` , który pobierze i wyświetli listę czytelników naszej aplikacji
- lista powinna mieć 2 stany: ukryty (wyświetla tylko informację o liczbie użytkowników) oraz rozwinięty (wyświetla listę użytkowników)
- pamiętaj o dodaniu przycisku, który będzie ukrywał i pokazywał listę
- na liście rozwiniętej powinny być widoczne takie dane jak imię, email, username, website
- użyj `useEffect` i `fetch` do pobrania danych
- obsłuż stan ładowania, błąd oraz anulowanie zapytania przez `AbortController`

Pytania po zadaniu

PYTANIE

Co powinno się wydarzyć, gdy starsze zapytanie zakończy się po nowszym?

PYTANIE

Jak odróżnić anulowanie operacji od błędu widocznego dla użytkownika?

PYTANIE

Które kombinacje stanów ładowania, danych i błędu powinny być niemożliwe?

Zadanie nr 13

20 minut

- napisz swój własny hook do pobierania danych z API
- hook powinien zwracać `data` , `isLoading` oraz `error`
- hook powinien przyjmować endpoint jako argument
- wykorzystaj go w widoku czytelników z poprzedniego zadania
- zadbaj o typ generyczny, np. `useApi<Reader[]>('/users')`
- na tym etapie bazowy adres API może być stałą wewnątrz hooka; przeniesiemy go do konfiguracji środowiska w zadaniu 16

Mockowanie API z MSW

- MSW przechwytuje realne requesty HTTP na poziomie przeglądarki lub środowiska testowego
- komponenty korzystają z tego samego kodu API co w produkcji
- mocki można współdzielić między developmentem, testami komponentów i testami E2E
- dobrze sprawdza się przy pracy równoległej z backendem

Problem server state

Użycie useEffect i fetch:

- brak cache
- brak retry
- brak refetch
- brak synchronizacji
- ręczne `isLoading`
- ręczne error handling

Czym jest TanStack Query?

- biblioteka do zarządzania server state
- oparta o hooki
- bardzo rozbudowana, ale można używać tylko tego, co potrzebujemy
- powstała jako React Query, ale teraz jest framework-agnostic (React, Vue, Solid, Svelte)
- w aplikacjach React najczęściej używana jako domyślna warstwa server state

Przykładowe funkcje:

- cache
- background refetch
- retry
- deduplikacja zapytań
- synchronizacja wielu komponentów

QueryClient

QueryClient to klasa, która zarządza cache'em i konfiguracją zapytań. Aby korzystać z React Query, musimy utworzyć instancję QueryClient i udostępnić ją naszej aplikacji za pomocą QueryClientProvider.

Przykładowy kod:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient();

root.render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

useQuery

Przykładowy kod:

```
import { useQuery } from '@tanstack/react-query';

const fetchUsers = async () => {
  const res = await fetch('/users');
  return res.json();
};

const Users = () => {
  const { data, isPending, isError, error } = useQuery({
    queryKey: ['users'],
    queryFn: fetchUsers,
  });

  if (isPending) return <p>Loading...</p>;
  if (isError) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
};
```

Jako zewnętrzny hook

Przykładowy kod:

```
import { useQuery } from '@tanstack/react-query';

export const useUsers = () => {
  return useQuery({
    queryKey: ['users'],
    queryFn: async () => {
      const res = await fetch('/users');
      return res.json();
    },
  });
};
```

Caching

- wyniki zapytań są przechowywane w cache i współdzielone pomiędzy obserwatorami
- dane są domyślnie uznawane za nieaktualne (`stale`) natychmiast po pobraniu
- nowa instancja zapytania może od razu pokazać dane z cache i jednocześnie odświeżyć je w tle
- nieaktywne zapytanie jest domyślnie usuwane z pamięci po 5 minutach (`gcTime`)

Stale Time vs Cache Time

staleTime

- jak długo dane są uznawane za "świeże"
- w tym czasie nie ma automatycznego refetch

gcTime (`cacheTime` przed v5)

- jak długo dane pozostają w pamięci po unmount

Przykładowy kod:

```
useQuery({
  queryKey: ['users'],
  queryFn: fetchUsers,
  staleTime: 1000 * 60, // 1 minuta
});
```


21.10.2026 19:17

Automatyczny Background Refetch

Refetch on window focus

- powrót do zakładki = dane się odświeżają
- reconnect internetu = dane się odświeżają
- zamontowanie nowej instancji query = nieaktualne dane mogą zostać odświeżone w tle

Automatyczny refetch dotyczy danych uznanych za `stale`. Najczęściej ograniczamy jego częstotliwość przez świadome ustawienie `staleTime`, zamiast wyłączać wszystkie triggery.

Przykładowy kod:

```
useQuery({
  queryKey: ['users'],
  queryFn: fetchUsers,
  refetchOnWindowFocus: false,
});
```

useMutation – coś więcej niż POST

- `useMutation` to hook, który pozwala na wykonywanie operacji mutujących dane (np. POST, PUT, DELETE) w sposób łatwy i zorganizowany
- w przeciwieństwie do `useQuery`, `useMutation` nie jest automatycznie wywoływany przy renderze komponentu, ale jest wywoływany ręcznie (np. w odpowiedzi na zdarzenie, takie jak kliknięcie przycisku)

Przykładowy kod:

```
const queryClient = useQueryClient();

const mutation = useMutation({
  mutationFn: addUser,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['users'] });
  },
});
```

Użycie w komponencie:

Przykładowy kod:

```
<button onClick={() => mutation.mutate({ name: 'New User' })}>
  Add User
</button>
```

Zadanie dodatkowe — TanStack Query

15 minut

- zainstaluj TanStack Query w projekcie `npm install @tanstack/react-query`
- dodaj `QueryClientProvider` do drzewa komponentów, aby udostępnić klienta zapytań w całej aplikacji
- przepisz pobieranie czytelników z API, aby korzystało z hooka `useQuery` z TanStack Query
- dodaj obsługę stanu ładowania i błędów, wykorzystując `isPending`, `isError` oraz `error` zwrócone z `useQuery`
- sprawdź w React Query Devtools lub w zakładce Network różnicę pomiędzy odczytem danych z cache a odświeżeniem w tle
- zostaw wcześniejszy własny hook jako punkt porównania, ale nie używaj go już w widoku czytelników

Router

- pozwala zastąpić renderowany komponent na widoku w zależności od bieżącego adresu URL w przeglądarce
- operuje na obiekcie `location` (oczywiście dostosowanego do potrzeb aplikacji SPA)
- zezwala na dodanie parametrów do ścieżek
- zezwala na wykorzystywanie dynamicznych linków (bazujących na zmiennych)
- routing powinien zachowywać dostępność linków i nawigacji klawiaturą

Przykładowy kod:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router";

import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Outlet

- `Outlet` wskazuje miejsce, w którym React Router wyrenderuje dopasowany route potomny
- używamy go w layoucie, gdy kilka ścieżek ma wspólny nagłówek, stopkę, menu albo strukturę strony
- layout pozostaje na ekranie, a zmienia się tylko zawartość wyrenderowana w `Outlet`
- bez `Outlet` route potomne mogą być dopasowane, ale nie będą miały gdzie się wyrenderować

Przykładowy kod:

```
import { Outlet } from 'react-router';

function Layout() {
  return (
    <>
      <Header />
      <main>
        <Outlet />
      </main>
      <Footer />
    </>
  );
}
```

Przykładowy kod:

```
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<Home />} />
    <Route path="about" element={<About />} />
    <Route path="dashboard" element={<Dashboard />} />
    <Route path="post/:id" element={<Post />} />
    <Route path="*" element={<NoMatch />} />
  </Route>
</Routes>
```

Przykładowy kod:

```
<nav>
  <Link to="/">Home</Link>
  <Link to="/about">About</Link>
  <Link to="/dashboard">Dashboard</Link>
  <Link to={`/post/${id}`}>New post</Link>
</nav>
```

Przykładowy kod:

```
const Post = () => {
  const { id } = useParams()

  return (
    <div>
      Post ID: {id}
    </div>
  )
}
```

Przykładowy kod:

```
const navigate = useNavigate();

navigate(`/transaction/${id}`);
```

Tryby React Router

Tryb	Charakter
Declarative	`BrowserRouter`, `Routes`, `Route` i standardowy routing komponentów
Data	konfiguracja tras poza renderem, loadery, actions, pending state i `createBrowserRouter`
Framework	Vite plugin, route modules, automatyczny code splitting oraz strategie SPA/SSR/SSG

W ćwiczeniach zaczynamy od Declarative Mode. Data i Framework Mode rozszerzają ten model, gdy aplikacja potrzebuje zintegrowanej obsługi danych lub renderowania.

React Router Data Mode

Przykładowy kod:

```
import { createBrowserRouter, RouterProvider } from 'react-router';

import Layout from './Layout';
import Home from './Home';
import BookDetails from './BookDetails';
import ErrorPage from './ErrorPage';

const homeLoader = () => fetch('/api/homeData')
  .then((res) => res.json());

const router = createBrowserRouter([{
  path: "/",
  element: <Layout />,
  children: [
    { index: true, element: <Home /> },
    { path: "books/:id", element: <BookDetails /> },
  ],
  loader: homeLoader,
  errorElement: <ErrorPage />,
}]);

export const App = () => <RouterProvider router={router} />;
```

Zadanie nr 14

30 minut

- zainstaluj React Router `npm i react-router`
- dodaj `BrowserRouter` w `src/main.tsx`
- przygotuj ścieżki dla: strony głównej (informacji o tym, co to za aplikacja), listy książek, pojedynczej książki i formularza kontaktowego
- strona główna powinna mieć ścieżkę `/`
- lista książek `/books`
- pojedyncza książka `/books/:id`
- formularz kontaktowy `/contact`
- przenieś listę książek z `App` do widoku `BooksPage`
- przygotuj wspólny `Layout` z nagłówkiem, stopką i miejscem na zagnieżdżone widoki
- użyj nested routes oraz komponentu `<Outlet />`
- dodaj nawigację w nagłówku aplikacji wykorzystując komponent `<Link>` z `react-router`

Przykładowy kod:

```
<Link to="/">Home</Link>
```

CSS inline

Przykładowy kod:

```
const User = ({ name }) => (
  <div style={{
    display: 'flex',
    color: 'blue'
  }}>
    <h2 style={{
      fontSize: '1.1rem',
      textTransform: 'uppercase'
    }}>{name}</h2>
  </div>
)
```

Stylowanie w praktyce

- wybierz jeden podstawowy sposób stylowania dla aplikacji
- CSS Modules są dobrym domyślnym wyborem w Vite i prostych aplikacjach React
- biblioteki komponentów, np. MUI, rozwiązują część UI, ale narzucają własny system stylów
- utility CSS, np. Tailwind, dobrze działa przy szybkim składaniu widoków i design systemach

ICSS

- w 2015 roku wprowadzono rozszerzenie do specyfikacji CSS, którego głównym zadaniem było umożliwienie ograniczenia zasięgu styli do poziomu komponentu, jednocześnie nadal pozostawiając selektory dostępne z poziomu zasięgu globalnego
- rozszerzenie to nazwano Interoperable CSS (ICSS)
- ICSS w zasadzie dodał dwa pseudoselektory CSS `:import` oraz `:export`
- głównym zadaniem była możliwość wyeksportowania na zewnątrz styli i zaimportowania ich w takich sposób, aby udostępnione style były dostępne jako obiekt z lokalnymi aliasami jako jego właściwościami
- lokalny alias to po prostu nazwa składająca się z liter i cyfr
- otworzyło to drogę do rozwoju CSS Modules

CSS Modules

- prosty sposób deklarowania styli lokalnych dla komponentów, dobrze wspierany przez Vite
- pozwala na wykorzystanie preprocessor'ów takich jak Sass
- pliki ze stylami są traktowane jako moduły
- każdy komponent posiada swój własny plik / moduł ze stylami
- nazwa każdego modułu ze stylami powinna być zakończona rozszerzeniem `.module.${ext}`
- bardzo łatwy sposób na enkapsulacje styli (enkapsulacja za pomocą generowane ID dodawanego do nazw klas)
- bardzo dobra adaptacja w przeglądarce - nie ma większych problemów z wydajnością, ponieważ docelowo style są traktowane tak samo jak zwykłe style css
- klasy css są dostępne na poziomie komponentu w obiekcie ze stylami
- style globalne powinny być trzymane w osobnym pliku / katalogu

Przykładowy kod:

```
import styles from './Counter.module.scss';

const Counter = ({ days, nextItemName }: CounterProps) => {
    return (
        <div className={styles.counter}>
            <div className={styles.counterText}>
                <span className={styles.nextTitle}>Następny {nextItemName}</span>
                <span className={styles.remainingDays}>{prepareDaysLabel()}</span>
            </div>
            <div className={styles.icon}>
                <Image src={calendar || `/icons/calendar.svg`} width={24} height={24} alt=""/>
            </div>
        </div>
    )
}

export default Counter;
```

JSS

- pierwsza próba tworzenia styli z wykorzystaniem Javascript
- koncepcja została zaproponowana i przygotowana w 1996 roku przez firmę Netscape (wówczas to rozwiązanie nie uzyskało wystarczająco dużego wsparcia przeglądarek i upadło, funkcjonowało wówczas pod nazwą JSSS)
- współcześnie JSS jest podobnym konceptem trzymania CSS w JS za pomocą obiektów Javascriptowych
- zasadność wykorzystania JSa do tworzenia CSS opiera się na dużej reużywalności obiektów i sporymi możliwościami enkapsulacji styli
- JSS nie jest powiązany z żadnym frameworkiem
- aby używać JSS w React możemy użyć pakietu react-jss

Przykładowy kod:

```
jss.setup(preset())
const styles = {
  '@global': {
    a: {
      textDecoration: 'underline'
    }
  },
  withTemplates: `
    background-color: green;
  `,
  button: {
    fontSize: 12,
    '&:hover': {
      background: 'blue'
    }
  },
  ctaButton: {
    extend: 'button',
    '&:hover': {
      background: color('blue').darken(0.3).hex()
    }
  },
  '@media (min-width: 1024px)': {
    button: {
      width: 200
    }
  }
}
```

Styled components

- bazuje na rozwiązaniach JSS
- biblioteka do tworzenia styli jako komponentów
- wykorzystuje Javascriptowy mechanizm tagged templates
- styled komponent może być tworzony jako dedykowany dla innego komponentu React lub jako samodzielny działający dla całej aplikacji
- zachowują się i działają w sposób taki jak inne elementy JSX
- zezwala na rozszerzania komponentów React poprzez dodawania styli do nich
- styled komponenty są reużywalne, mogą korzystać z propsów i trzymać logikę powiązaną z zarządzaniem stylami

Przykładowy kod:

```
import styled from 'styled-components';

const Header = styled.div`
  background-color: #ddd;
  color: blue;
  border-bottom: 2px solid;
  padding: ${({ padding }) => `${padding}px`}
`


// usage

<Header padding={20}/>
```

Przykładowy kod:

```
const functionA = value => {
  return value[0] + 'string';
}

functionA`test`; // teststring
```


21.10.2026 19:17

Czym są Tagged Templates?

- zaawansowana funkcja w JS, która umożliwia manipulację literalami szablonowymi (template literals) za pomocą specjalnych funkcji zwanych tag functions
- dzięki temu możemy dynamicznie przekształcać ciągi znaków lub fragmenty kodu

Przykładowy kod:

```
function myTag(strings, ...values) {
  // strings: tablica stałych fragmentów szablonu
  // values: tablica interpolowanych wartości
  return `Modified: ${strings.join('')} | ${values.join(', ')}`;
}

const name = "Alice";
const age = 25;

const result = myTag`Name: ${name}, Age: ${age}`;
console.log(result); // Modified: Name: , Age: | Alice, 25
```

Tagged templates w praktyce

Tagged templates pozwalają na manipulację szablonem przed jego ostatecznym wygenerowaniem, np. sanitizację danych, translację czy logowanie.

Przykładowy kod:

```
function sanitize(strings, ...values) {
  return strings.reduce((result, str, i) => {
    const safeValue = String(values[i] || '').replace(/</g, "&lt;").replace(/>/g, "&gt;");
    return result + str + safeValue;
  }, '');
}

const userInput = "<script>alert('x')</script>";
const output = sanitize`User input: ${userInput}`;
console.log(output); // User input: &lt;script&gt;alert('x')&lt;/script&gt;
```

Tailwind

- framework wydany w 2019 roku
- stylowanie za pomocą atomowych klas CSS, tzw. utility classes
- pisanie kodu CSS jest szybkie i intuicyjne
- brak predefiniowanych komponentów, opiera się o tzw. Utility classes

Najważniejsze cechy Tailwind CSS

- **podejście Utility-First** - nie piszemy własnych klas css, korzystamy tylko z gotowych odpowiedzialnych za konkretne działanie, np (`text-center`)
- **elastyczność** - nie mamy narzuconego konkretnego wyglądu
- **łatwość konfiguracji** - cała konfiguracja w jednym pliku `tailwind.config.js` , gdzie możemy zdefiniować też własne kolory itp.
- **responsywność** - Tailwind dostarcza wbudowane klasy odpowiedzialne za responsywność, np.: `md:text-xl`
- **jakość kodu** - brak nadmiarowego kodu (zombie css)
- **integracja z narzędziami** - Tailwind działa świetnie z frameworkami takimi jak React, Vue czy Angular
- **ekosystem** - rozbudowana dokumentacja i bogata społeczność, ponadto istnieją dodatkowe rozszerzenia, takie jak Tailwind UI (płatne komponenty), czy pluginy wspierające rozwój aplikacji

21.10.2026 19:17

Przykładowy kod:

```
export default function App() {
  return (
    <div className="text-center">
      <h1 className="text-3xl font-bold underline">
        Hello, world!
      </h1>
    </div>
  );
}
```

Przykładowy kod:

```
// tailwind.config.js
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Przykładowy kod:

```
/* src/index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Zadanie nr 15

30 minut

- dodaj do projektu Material UI: https://mui.com/material-ui/getting-started/

Przykładowy kod:

```
npm install @mui/material @emotion/react @emotion/styled
```

- jeśli używasz Styled Components, zainstaluj też `styled-components`

Przykładowy kod:

```
npm install styled-components
```

- z wykorzystaniem MUI dodaj komponent `Avatar`, aby wyświetlić zdjęcie użytkownika `https://placehold.co/150`
- za pomocą `CSS modules` ostyluj listę książek
- za pomocą `Styled Components` ostyluj nawigację, stopkę oraz zmień styl awatara tak, aby otrzymał dodatkową ramkę wkoło
- zostaw style lokalne przy komponentach, których dotyczą

.env

- w projekcie Vite możemy deklarować i używać zmiennych środowiskowych
- nazwy zmiennych środowiskowych dostępnych w aplikacji powinny być prefiksowane za pomocą VITE_
- zadeklarowane zmienne nie powinny przechowywać sekretów
- nigdy nie trzymaj sekretów w repozytorium
- pamiętaj, że wartości ze zmiennych środowiskowych w komponentach klienckich zostaną przeniesione do kodu produkcyjnego

Rodzaje plików .env

Plik	Opis
.env	Zasadniczy plik ze zmiennymi
.env.development	Używany w środowisku deweloperskim
.env.production	Używany podczas produkcyjnego builda
.env.test	Używany podczas testów
.env.local	Nadpisuje wartości w innych .env plikach dla lokalnego środowiska
.env.development.local	Specyficzny dla lokalnego środowiska deweloperskiego
.env.production.local	Specyficzny dla lokalnych ustawień produkcyjnych

Nigdy nie trzymaj sekretów w aplikacji React — przede wszystkim prywatnych kluczy API.

Przykładowy kod:

```
# .env.local
VITE_API_URI=http://localhost:3000
```

Przykładowy kod:

```
const buildUrl = (endpoint: string): string =>
  `${import.meta.env.VITE_API_URI}/${endpoint}`;
```

Zadanie nr 16

20 minut

- dodaj plik `.env.local`
- zadeklaruj zmienną środowiskową `VITE_API_URI` z wartością `https://jsonplaceholder.typicode.com`
- przebuduj własny hook do wykonywania zapytań tak, aby pobierał bazowy adres z `import.meta.env.VITE_API_URI`
- do hooka przekazuj ścieżkę, np. `/users`, oraz opcjonalne źródło: `api` albo `public`; domyślnym źródłem powinno być `api`
- wydziel budowanie pełnego adresu do funkcji `src/utils/buildResourceUrl.ts`, którą wykorzystasz później w testach
- dla źródła `api` funkcja powinna dodać bazowy adres z `VITE_API_URI`, a dla `public` pozostawić lokalną ścieżkę bez zmian
- sprawdź oba warianty, pobierając czytelników przez `useApi<Reader[]>('/users')` oraz przygotowany plik przez `useApi<BookItem[]>('/books.json', 'public')`

Zarządzanie uprawnieniami

Ochrona trasy po stronie klienta steruje interfejsem i nawigacją, ale nie jest granicą bezpieczeństwa. Backend musi niezależnie sprawdzić uwierzytelnienie i uprawnienia dla każdego chronionego żądania.

Przykładowy kod:

```tsx
enum Roles {
  ADMIN = 'admin',
  USER = 'user',
}

interface ProtectedRouteType<T = {}> {
  children: ReactNode;
  necessaryRole?: Roles;
  userRole?: Roles;
  auth: boolean;
}

const ProtectedRoute = ({ userRole, auth, necessaryRole, children }: ProtectedRouteType) => {
  const canActivate = () => !!(auth === true && userRole === necessaryRole);

  if (!canActivate()) {
    return <Navigate to="/" replace />
  }

  return <>{children}</>;
}
```

Przykładowy kod:

```
<Routes>
<Route path="/" index element={<HomePage />}/>
  <Route path="/history" element={<TransactionsPage />}/>
  <Route path="/protected" element={
     <ProtectedRoute auth={false}>
         <TransactionsPage />
     </ProtectedRoute>
 } />
</Routes>
```

Zadanie nr 17

25 minut

- przygotuj zabezpieczoną ścieżkę dla widoku z czytelnikami (jeśli nie masz takiego widoku dodaj nowy `route`)
- dodaj tymczasowy stan logowania w aplikacji, np. `isAuthenticated`
- jeśli nastąpi próba wejścia na widok czytelników bez uprzedniego zalogowania, przekieruj użytkownika na widok z informacją, że nie ma uprawnień do przeglądania tej strony
- po zalogowaniu użytkownik powinien móc wejść na zabezpieczony widok
- jeśli korzystasz z `createBrowserRouter`, dodaj `errorElement` dla błędów routingu
- potraktuj to wyłącznie jako ochronę interfejsu: backend nadal musi niezależnie sprawdzać uwierzytelnienie i uprawnienia dla każdego żądania

Pytania po zadaniu

PYTANIE

Przed którymi działaniami nie chroni zabezpieczenie trasy w interfejsie?

PYTANIE

Co backend musi zweryfikować niezależnie dla każdego żądania?

PYTANIE

Jak zachować zamiar użytkownika po zalogowaniu, nie ufając ślepo adresowi przekierowania?

Portale

- mechanizm, który zezwala na renderowanie komponentów React poza miejscem w strukturze, gdzie powinny one zostać wyrenderowane
- zezwala na referowanie elementów w dowolnym miejscu w DOM lub w oknach zależnych (new Window)
- portale mogą być użyteczne, gdy stylowanie komponentów ogranicza możliwości poprawnego renderowania komponentów, np. gdy komponent ma ustawiony `z-index` lub `overflow` z wartością `hidden`
- element renderowany za pośrednictwem Portalu jest nadal pod kontrolą React
- komponenty renderowane w Portalu mogą współdzielić stan, mają dostęp do przekazywanych propsów i do context'u
- event bubbling (bąbelkowanie) będzie działało w ten sam sposób jak bez Portalu, dlatego zdarzenia odpalone wewnątrz Portalu będą propagowaną w górę drzewa React

Przykładowy kod:

```
import { createPortal } from 'react-dom';

const Modal = ({ children }) => {
  return createPortal(
    <div>{children}</div>,
    document.getElementById('modal'),
  )
}
```

Przykładowy kod:

```
<body>
  <div id="app"></div> /* React app */
  <div id="modal"></div> /* Portal */
</body>
```

Zadanie nr 18

20 minut

- w pliku `index.html` dodaj element, do którego wyrenderujesz modal, np. `<div id="modal-root"></div>`
- z użyciem React Portal przygotuj modal, który będzie wyświetlał się po kliknięciu "Zaloguj"
- logowanie odbędzie się dopiero po kliknięciu "Zaloguj" w modalu
- oznacz modal semantycznie przez `role="dialog"` , `aria-modal="true"` i dostępną nazwę
- po otwarciu przenieś focus do modala i nie pozwól, aby nawigacja klawiaturą wychodziła poza jego interaktywne elementy
- umożliw zamknięcie modala klawiszem Escape
- po zamknięciu modala focus powinien wrócić do przycisku otwierającego

21.10.2026 19:17

Pytania po zadaniu

PYTANIE

Co portal zmienia w DOM, a czego nie zmienia w drzewie Reacta?

PYTANIE

Gdzie powinien trafić focus, jeśli element otwierający przestał istnieć?

PYTANIE

Jak sprawdzić obsługę modala bez polegania wyłącznie na myszy?

Context API

- Context pozwala udostępnić wartość niżej w drzewie bez przekazywania jej przez każdy poziom propsów
- dobrze sprawdza się dla konfiguracji aplikacji, motywu, języka, aktualnego użytkownika lub zależności typu klient API
- nie jest pełnym storem i nie rozwiązuje sam z siebie modelowania złożonych zmian stanu
- jeśli problemem jest tylko kilka poziomów prop drillingu, często lepsza będzie kompozycja komponentów
- każdy komponent używający `useContext` zostanie ponownie wyrenderowany, kiedy zmieni się referencja wartości Providera

Przykładowy kod:

```
type Theme = 'light' | 'dark';

type ThemeContextValue = {
  theme: Theme;
  toggleTheme: () => void;
};

const ThemeContext = createContext<ThemeContextValue | null>(null);
```

Przykładowy kod:

```tsx
function ThemeProvider({ children }: PropsWithChildren) {
  const [theme, setTheme] = useState<Theme>('light');

  const value = useMemo(
    () => ({
      theme,
      toggleTheme: () => {
        setTheme((currentTheme) =>
          currentTheme === 'light' ? 'dark' : 'light'
        );
      },
    }),
    [theme]
  );

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Przykładowy kod:

```
function useTheme() {
  const context = useContext(ThemeContext);

  if (!context) {
    throw new Error('useTheme must be used inside ThemeProvider');
  }

  return context;
}

function ThemeSwitch() {
  const { theme, toggleTheme } = useTheme();

  return <button onClick={toggleTheme}>{theme}</button>;
}
```

Context API – na co uważać?

- Context propaguje zmianę po referencji wartości, dlatego obiekty i funkcje warto stabilizować albo rozdzielać na mniejsze konteksty
- dla często zmienianych wartości warto rozważyć dedykowany store z selektorami
- Context dobrze przenosi wartość, ale reguły zmiany stanu nadal musimy zaprojektować sami
- reducer + context może wystarczyć w średnich przypadkach, ale wymaga pilnowania podziału danych i akcji
- zbyt wiele kontekstów bez jasnych granic utrudnia śledzenie przepływu danych

Zadanie nr 19

25 minut

- w naszej aplikacji chcemy obsłużyć dwa motywy: `dark` i `light`
- przygotuj `ThemeContext`, który udostępni aktualny motyw i funkcję jego zmiany
- dodaj przycisk w nagłówku do zmiany motywu
- w stopce dodaj informację o aktualnie wybranym motywie
- zadbaj o to, żeby hook używany poza providerem zwracał czytelny błąd

Kompozycja

- kluczowy wzorzec projektowy w React, pozwala na tworzenie złożonych komponentów poprzez łączenie mniejszych, wielokrotnego użytku
- React preferuje kompozycję zamiast dziedziczenia (które jest częstsze w tradycyjnych OOP)
- kompozycja sprzyja elastyczności i modularności aplikacji

Główne założenia kompozycji:

- komponenty mogą być używane jako dzieci innych komponentów lub jako ich elementy składowe
- dzięki przekazywaniu dzieci (`props.children`) lub funkcji, komponenty rodziców mogą definiować, jak komponenty dzieci powinny być renderowane

props.children

Przykładowy kod:

```
function Card({ children }) {
  return (
    <div style={{ border: '1px solid #ddd', padding: '10px', borderRadius: '5px' }}>
      {children}
    </div>
  );
}

function App() {
  return (
    <Card>
      <h2>Tytuł</h2>
      <p>To jest zawartość karty.</p>
    </Card>
  );
}
```

render props, czyli funkcje renderujące

Przykładowy kod:

```
function DataFetcher({ render }) {
  const data = ['Element 1', 'Element 2', 'Element 3'];
  return <div>{render(data)}</div>;
}

const App = () => (
  <DataFetcher
    render={(data) => (
      <ul>
        {data.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    )}
  />
)
```

Wzorzec „Slot"

Przykładowy kod:

```
function Layout({ header, content, footer }) {
  return (
    <div>
      <header>{header}</header>
      <main>{content}</main>
      <footer>{footer}</footer>
    </div>
  );
}

function App() {
  return (
    <Layout
      header={<h1>Nagłówek</h1>}
      content={<p>To jest główna treść.</p>}
      footer={<small>Stopka</small>}
    />
  );
}
```

Higher Order Component

Przykładowy kod:

```
function withLogger(Component) {
  return function WrappedComponent(props) {
    console.log('Renderowanie komponentu z props:', props);
    return <Component {...props} />;
  };
}

function MyComponent({ message }) {
  return <p>{message}</p>;
}

const MyComponentWithLogger = withLogger(MyComponent);

function App() {
  return <MyComponentWithLogger message="Cześć!" />;
}
```

Code splitting

- wraz z rozrastającym się kodem aplikacji, rośnie również rozmiar plików produkcyjnych
- Code Splitting jest mechanizmem wspieranym przez takie narzędzia jak: Webpack, Rollup, Browserify
- Code Splitting zezwala na tworzenie dużej ilości małych paczek kodu, które mogą być ładowane na żądanie w trakcie działania aplikacji
- dobrym miejscem na podział aplikacji za pomocą Code Splittingu są ścieżki w routerze
- sposoby na Code splitting w aplikacji React: `import()` oraz `React.lazy()`

Przykładowy kod:

```
const Home = lazy(() => import('./routes/Home'));
const About = lazy(() => import('./routes/About'));

const App = () => (
  <Router>
    <Suspense fallback={<div>Loading…</div>}>
      <Routes>
        <Route path="/" element={<Home />}/>
        <Route path="/about" element={<About />}/>
      </Routes>
    </Suspense>
  </Router>
);
```

Default export vs named export

Komponent, który importujemy należy eksportować domyślnie. Wówczas składnia importu jest prostsza.

Brak eksportu domyślnego powoduje błąd, ponieważ `lazy` działa tylko z eksportami domyślnymi. Poniższy przykład pokazuje jak zmapować `named export` na `default export`, gdy musimy zachować `named export`.

Przykładowy kod:

```
const Home = lazy(() =>
  import('./routes/Home').then((module) => ({ default: module.Home }))
);
```

Przykładowy kod:

```
function ImportExample() {
  const handleClick = async () => {
    try {
      const { moduleA } = await import('./moduleA');
      moduleA();
    } catch (error) {
      console.error('Nie udało się załadować modułu', error);
    }
  };

  return <button onClick={handleClick}>Load</button>;
}
```

Przykładowy kod:

```
import React, { Suspense } from 'react';

const OtherComponent = React.lazy(() => import('./OtherComponent'));
const AnotherComponent = React.lazy(() => import('./AnotherComponent'));

function MyComponent() {
  return (
    <div>
      <Suspense fallback={<div>Loading...</div>}>
        <section>
          <OtherComponent />
          <AnotherComponent />
        </section>
      </Suspense>
    </div>
  );
}
```

Zadanie nr 20

15 minut

- wykorzystując code splitting podziel aplikację według widoków routingu
- użyj `React.lazy` oraz `<Suspense />`
- przygotuj fallback widoczny podczas ładowania widoku
- lazy loading zastosuj na poziomie stron, np. `BooksPage` , `BookDetailsPage` , `ContactPage`
- upewnij się, że błąd ładowania widoku nie psuje całej aplikacji - użyj route error boundary albo osobnego fallbacku błędu

Concurrent Features

- nazywany w poprzednich wersjach Concurrent Mode
- zestaw funkcji w React, które umożliwiają bardziej płynne i responsywne renderowanie interfejsu użytkownika
- zaprojektowane, aby poprawić wydajność aplikacji poprzez lepsze zarządzanie czasem potrzebnym na renderowanie oraz interakcje z użytkownikiem
- w skład wchodzą między innymi takie funkcjonalności jak: `Suspense`, `useTransition`, `useDeferredValue`

useTransition

- hook wprowadzony w React 18
- umożliwia zarządzanie przejściami stanu w aplikacjach, poprawiając wrażenie płynności i responsywności interfejsu użytkownika
- pozwala oznaczyć część aktualizacji stanu jako „niepilną” i przeprowadzić ją w tle, bez blokowania innych ważniejszych aktualizacji, takich jak responsywność interfejsu

Przykładowy kod:

```
const [isPending, startTransition] = useTransition();
```

- isPending - boolean, wskazujący, czy niepilne aktualizacje są w toku
- startTransition - funkcja, która otacza niepilne operacje.

Przykładowy kod:

```
import { useState, useTransition } from 'react';

function SearchableList({ items }) {
  const [filteredItems, setFilteredItems] = useState(items);
  const [isPending, startTransition] = useTransition();

  const handleInputChange = (e) => {
    const query = e.target.value;

    // Filtruj elementy w tle
    startTransition(() => {
      setFilteredItems(
        items.filter((item) => item.toLowerCase().includes(query.toLowerCase()))
      );
    });
  };

  return (
    <div>
      {isPending && <p>Updating list...</p>}
      <ul>
        {filteredItems.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}
```

Memoizacja

- technika optymalizacji w programowaniu polegająca na przechowywaniu wyników funkcji dla określonych zestawów argumentów, aby uniknąć wielokrotnego przeliczania tych samych wyników w przyszłości
- memoizacja poprawia wydajność, szczególnie w przypadku funkcji, które są kosztowne obliczeniowo lub są często wywoływane z tymi samymi argumentami
- przy pierwszym wywołaniu funkcji z danym zestawem argumentów, wynik obliczeń jest zapisywany w specjalnej strukturze danych, zazwyczaj w obiekcie
- przy kolejnych wywołaniach z tymi samymi argumentami funkcja nie wykonuje obliczeń, lecz zwraca zapisany wcześniej wynik

Gdzie przyda się memoizacja?

Poniższy kod nie jest zbyt efektywny. Funkcja `fibonacci` wykonuje się rekurencyjnie wiele razy z tymi samymi argumentami.

Dokładny czas zależy od środowiska. Zmierz przykład w używanej przeglądarce lub runtime.

Przykładowy kod:

```
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

console.log(fibonacci(40));
```

Po dodaniu memoizacji

Po dodaniu cache wykonaj ten sam pomiar i porównaj liczbę wywołań oraz czas wykonania.

Przykładowy kod:

```
function fibonacciMemo() {
  const memo = {};

  function fibonacci(n) {
    if (n in memo) return memo[n]; // Sprawdź, czy wynik jest zapisany
    if (n <= 1) return n;
    memo[n] = fibonacci(n - 1) + fibonacci(n - 2); // Zapisz wynik
    return memo[n];
  }

  return fibonacci;
}

const fibonacci = fibonacciMemo();
console.log(fibonacci(40));
```

Ręczna memoizacja

- `React.memo` może ograniczyć render komponentu, gdy jego propsy się nie zmieniły
- `React.memo` porównuje propsy za pomocą porównania płytkiego (shallow comparison)
- dla propsów referencyjnych (np. tablice, obiekty, funkcje) sprawdzana jest tylko ich referencja, a nie rzeczywista zawartość
- memoizacja jest kosztem: komplikuje kod i może maskować problem w architekturze stanu
- zaczynamy od pomiaru w Profilerze / Performance Tracks, nie od automatycznego owijania komponentów

Przykładowy kod:

```
const Component = ({ name }) => {
  return <h2>{name}</h2>
}

export const MemoizedComponent = React.memo(Component);
```

Drugi argument w memo

- drugi argument w funkcji `React.memo` to opcjonalna funkcja porównania (ang. comparison function)
- pozwala na bardziej zaawansowane kontrolowanie, kiedy komponent powinien być ponownie renderowany
- domyślnie `React.memo` porównuje tylko płytko (shallow comparison) poprzednie i nowe propsy, ale w niektórych przypadkach, zwłaszcza gdy propsy zawierają złożone obiekty lub funkcje, może być konieczne użycie własnej logiki porównania

Przykładowy kod:

```
const MyComponent = React.memo(
  (props) => {
    // renderowanie komponentu
  },
  (prevProps, nextProps) => {
    // funkcja porównania
    return prevProps.value === nextProps.value; // przykład porównania
  }
);
```

useMemo i useCallback

- `useMemo` służy do zapamiętywania (memoizacji) wyniku jakiejś obliczeniowej operacji
- przydatne przy kosztownych obliczeniach albo stabilnym kontrakcie referencji

Przykładowy kod:

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

- `useCallback` działa podobnie jak `useMemo`, ale zamiast zapamiętywać wynik obliczeń, zapamiętuje samą funkcję
- przydatne, gdy stabilna referencja funkcji jest częścią kontraktu komponentu albo hooka

Przykładowy kod:

```
const memoizedCallback = useCallback(() => { doSomething(a, b); }, [a, b]);
```

Zadanie nr 21

20 minut

- użyj React DevTools Profiler albo React Performance Tracks w Chrome DevTools, aby zidentyfikować kosztowny render lub obliczenie
- jeśli filtrowanie książek jest rzeczywiście kosztowne, użyj `useMemo` i ponownie wykonaj pomiar
- jeśli niestabilny callback powoduje ponowne renderowanie zmemoizowanego komponentu, ustabilizuj go przez `useCallback` i porównaj wynik
- jeżeli pomiar nie pokazuje problemu, pozostaw kod bez dodatkowej memoizacji i zapisz ten wniosek

Pytania po zadaniu

PYTANIE

Który pomiar stanowi wiarygodny punkt odniesienia przed optymalizacją?

PYTANIE

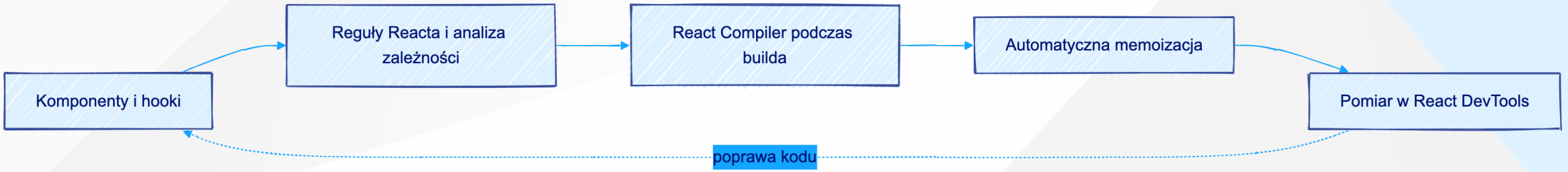
Jakie koszty lub ryzyko nieaktualnych danych wprowadza memoizacja?

PYTANIE

Kiedy wynik pomiaru uzasadnia pozostawienie prostszego kodu?

React Compiler

- automatyzuje część memoizacji na etapie builda
- analizuje komponenty i hooki, zakładając że są czyste i zgodne z regułami Reacta
- zmniejsza potrzebę ręcznego używania `React.memo`, `useMemo` i `useCallback`
- optymalizuje przede wszystkim aktualizacje już wyrenderowanego UI
- nie zwalnia z profilowania - nadal trzeba najpierw zlokalizować realny problem wydajnościowy
- nie naprawia efektów ubocznych w renderze ani niestabilnego modelu stanu


Komponenty i hooki
Reguły Reacta i analiza zależności
React Compiler podczas builda
Automatyczna memoizacja
Pomiar w React DevTools
poprawa kodu

Dyrektywy React Compiler

Przykładowy kod:

```
function BookCard({ book }: Props) {
  "use memo";

  return <article>{book.title}</article>;
}
```

- `"use memo"` może jawnie oznaczyć funkcję do optymalizacji przez Compiler
- `"use no memo"` pozwala tymczasowo wyłączyć optymalizację dla konkretnej funkcji
- dyrektywy są mechanizmem stopniowego wdrażania i diagnostyki, a nie domyślnym stylem kodu
- zachowanie `"use memo"` zależy od trybu kompilacji: `annotation`, `infer` albo `all`
- przy `"use no memo"` dokumentuj przyczynę i warunek usunięcia wyjątku

React Performance Tracks

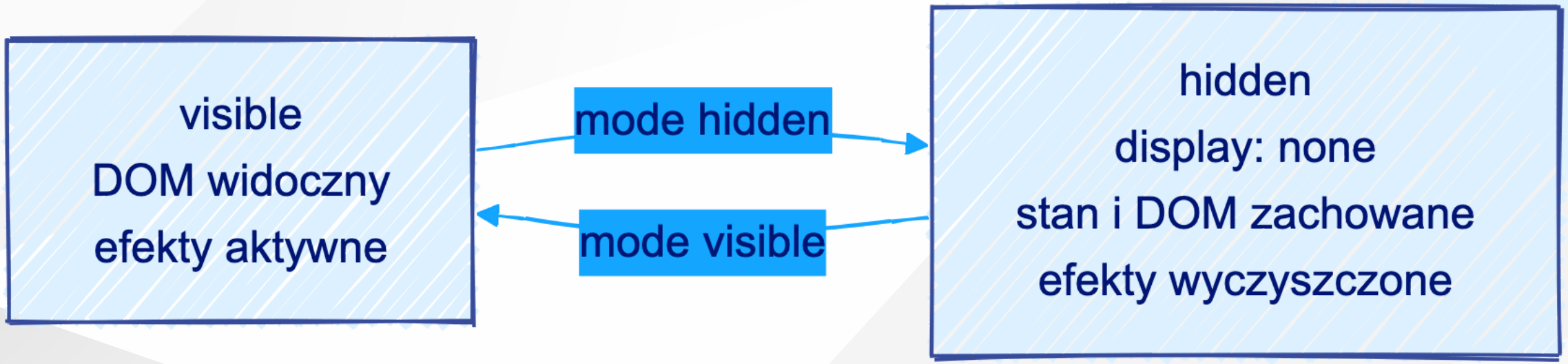
- dodatkowe ścieżki Reacta widoczne w profilu Performance w Chrome DevTools
- ułatwia rozróżnienie problemu Reacta od problemu JavaScriptu, layoutu albo sieci
- dobry punkt startowy przed dodaniem memoizacji

Activity

- `<Activity>` pozwala ukryć część UI bez utraty wewnętrznego stanu dzieci
- w trybie `hidden` React ustawia `display: none`, czyści efekty i odkłada aktualizacje
- stan i DOM pozostają dostępne do późniejszego przywrócenia
- ukryty widok może zostać wstępnie wyrenderowany z niższym priorytetem
- przydaje się dla paneli, sidebarów, zakładek i widoków, do których użytkownik często wraca

Przykładowy kod:

```
import { Activity } from 'react';

<Activity mode={isOpen ? 'visible' : 'hidden'}>
  <Sidebar />
</Activity>
```


visible
DOM widoczny
efekty aktywne
mode hidden
mode visible
hidden
display: none
stan i DOM zachowane
efekty wyczyszczone

Activity – wybór mechanizmu

Mechanizm	Stan	DOM	Efekty po ukryciu
renderowanie warunkowe	tracony	usuwany	cleanup
CSS `display: none`	zachowany	zachowany	nadal aktywne
`<Activity mode="hidden">`	zachowany	zachowany	cleanup

Elementy DOM z własnym zachowaniem, np. `video`, mogą wymagać jawnego cleanupu nawet wewnątrz `Activity`.

ViewTransition

- `<ViewTransition>` służy do animowania przejść komponentów razem z Transition i Suspense
- API jest stabilne od React 19.3 i obecnie działa w środowisku DOM
- React koordynuje natywne View Transitions dla zmian uruchamianych przez Transition
- animacja otacza zmianę stanu, ale nie zastępuje modelu stanu ani routingu
- projektuj wariant bez animacji dla `prefers-reduced-motion`

Przykładowy kod:

```
import { ViewTransition } from 'react';

<ViewTransition>
  <BookDetails />
</ViewTransition>
```

Poprzedni widok
Transition / Suspense / useDeferredValue
Migawki przed i po zmianie DOM
Animacja View Transition
Następny widok

ViewTransition – kiedy użyć?

- gdy animacja pomaga zachować kontekst między dwoma stanami interfejsu
- gdy aktualizacja jest wywołana przez Transition, Suspense albo odroczoną wartość
- po sprawdzeniu wsparcia przeglądarek i zachowania bez animacji
- jako progresywne ulepszenie, jeśli brak animacji nie blokuje podstawowego działania interfejsu

<React.StrictMode>

- dodany do React w 2018 roku (pierwotnie dotyczył tylko komponentów klasowych)
- ma za zadanie pomagać deweloperom w aktualizacji aplikacji poprzez rezygnację ze starego, niewspieranego API
- dodaje zestaw podpowiedzi i ostrzeżeń, aby pomóc uniknąć typowych pułapek związanych z procesem developmentu
- pomaga wykrywać nieczyste funkcje i brakujące operacje cleanup
- dodatkowo wywołuje w development funkcje komponentów, efekty i callback refs, aby ujawnić błędy zależne od efektów ubocznych
- działa tylko w trybie deweloperskim
- nie zmienia zachowania produkcyjnego aplikacji

Unikaj pułapek w StrictMode

1. nie twórz efektów ubocznych w funkcji renderującej
   - efekty uboczne, takie jak modyfikacja DOM czy zapytania sieciowe, powinny być umieszczane w `useEffect`
2. używaj funkcji czyszczących
   - `useEffect` powinien zwracać funkcję czyszczącą, szczególnie w przypadku: Subskrypcji, Timerów, WebSocket
3. unikaj niekontrolowanego dostępu do zmiennych
   - każda zmienna, która jest używana w efekcie, powinna być dodana do listy zależności (`dependency array`).

React Dev Tools

- rozszerzenie przeglądarki oraz narzędzie developerskie
- pozwala na inspekcję i debugowanie aplikacji napisanych w React
- pozwala lepiej zrozumieć strukturę komponentów, ich stan oraz propsy
- przydatne przy monitorowaniu wydajności aplikacji

Wtyczki:

- Chrome: React Developer Tools for Chrome
- Firefox: React Developer Tools for Firefox

Globalne zarządzanie stanem upraszcza komunikację między komponentami, ale wprowadza koszt architektoniczny, wydajnościowy i poznawczy.

Współdzielony stan aplikacji

- stan potrzebny w wielu niezależnych miejscach aplikacji
- jedno źródło prawdy dla wybranego fragmentu client state
- reguły zmiany stanu zapisane poza pojedynczym komponentem
- komponenty odczytują tylko te fragmenty stanu, które są im potrzebne
- łatwiejsze debugowanie, gdy zmiany są jawne i możliwe do odtwórzenia

WAŻNE

Globalny store nie jest miejscem na każdy stan. Najpierw ustalamy, czy to UI state, client state czy server state.

Kiedy warto używać globalnego zarządzania stanem?

- client state jest współdzielony - koszyk, preferencje, dane sesji, uprawnienia, aktywne filtry
- zmiany mają reguły biznesowe - jedna akcja wpływa na kilka pól albo kilka widoków
- potrzebujemy selektorów - komponenty powinny reagować tylko na wybrany fragment stanu
- debugowanie zmian jest ważne - przydaje się historia akcji, DevTools albo middleware
- stan ma żyć dłużej niż widok - przejście między ekranami nie powinno resetować danych

Kiedy NIE używać globalnego zarządzania stanem?

- stan należy tylko do jednego komponentu lub jednego formularza
- problem da się rozwiązać przez kompozycję komponentów
- przechowujemy wynik zapytania do API, status ładowania albo błąd requestu - to zwykle server state
- biblioteka stanu ma zastąpić brak modelu danych i granic odpowiedzialności
- aplikacja jest mała, a dodatkowa warstwa utrudnia czytanie kodu

WAŻNE

Globalne zarządzanie stanem to KOSZT, a nie benefit.

21.10.2026 19:17

Jak wybrać mechanizm stanu?

Problem	Pierwszy wybór
stan używany tylko w komponencie	`useState`
kilka powiązanych akcji i reguł przejścia	`useReducer`
wartość potrzebna w wielu miejscach drzewa	Context
client state używany w wielu niezależnych widokach	Zustand / Redux Toolkit
cache, status i odświeżanie danych z API	TanStack Query
stan formularza	React Hook Form / Form Hooks

WAŻNE

Nie wybieramy biblioteki od stanu, dopóki nie wiemy, jaki typ stanu modelujemy.

Redux Toolkit

- współczesny standard pracy z Reduxem
- zastępuje ręczną konfigurację store'a, akcje i reducera pisane osobno
- używa Immer, więc w reducerach możemy pisać mutacyjny zapis, a stan pozostaje aktualizowany niemutowalnie
- dobrze sprawdza się, gdy potrzebujemy przewidywalnego client state, DevTools i jasnych reguł zmiany stanu
- w nowych aplikacjach nie zaczynamy od `createStore`, `connect` i ręcznych action creatorów

Przykładowy kod:

```
import { configureStore } from '@reduxjs/toolkit';
import { userReducer } from './userSlice';

export const store = configureStore({
  reducer: {
    user: userReducer,
  },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Przykładowy kod:

```
import { createSlice, type PayloadAction } from '@reduxjs/toolkit';

type UserState = {
  name: string | null;
};

const initialState: UserState = {
  name: null,
};

const userSlice = createSlice({
  name: 'user',
  initialState,
  reducers: {
    loggedIn(state, action: PayloadAction<string>) {
      state.name = action.payload;
    },
    loggedOut(state) {
      state.name = null;
    },
  },
});

export const { loggedIn, loggedOut } = userSlice.actions;
export const userReducer = userSlice.reducer;
```

Przykładowy kod:

```ts
import { useDispatch, useSelector } from 'react-redux';
import type { RootState, AppDispatch } from './store';

export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

Przykładowy kod:

```tsx
function UserMenu() {
  const name = useAppSelector((state) => state.user.name);
  const dispatch = useAppDispatch();

  if (!name) {
    return (
      <button onClick={() => dispatch(loggedIn('Ada'))}>
        Zaloguj
      </button>
    );
  }

  return <button onClick={() => dispatch(loggedOut())}>{name}</button>;
}
```

Redux Toolkit vs Context

Context	Redux Toolkit
część Reacta	zewnętrzna biblioteka
przenosi wartość przez drzewo	modeluje store, akcje i reguły zmian
dobry dla konfiguracji, motywu, języka, usera	dobry dla większego client state
brak natywnych selektorów	selektory i dobre DevTools
prosty start	większy koszt architektoniczny

Zadanie nr 22

30 minut

- zainstaluj Redux Toolkit i React Redux

Przykładowy kod:

```
npm install @reduxjs/toolkit react-redux
```

- skonfiguruj Redux Toolkit w naszej aplikacji
- dodaj `Provider` z `react-redux` w `src/main.tsx`
- przygotuj slice do zarządzania zalogowanym użytkownikiem
- przygotuj typowane hooki `useAppSelector` i `useAppDispatch`
- użyj danych użytkownika w nagłówku oraz w stopce
- do wysłania akcji logowania i wylogowania użyj `useAppDispatch`
- nie przenoś do Reduxa danych pobieranych przez TanStack Query

Zadanie nr 23

20 minut

- dodaj selektor zwracający informację, czy użytkownik jest zalogowany
- dodaj selektor zwracający nazwę użytkownika albo tekst `Gość`
- użyj selektorów w nagłówku i stopce zamiast odczytywać cały fragment stanu
- sprawdź w Redux DevTools, jakie akcje zostały wysłane podczas logowania i wylogowania
- sprawdź, które komponenty renderują się po zmianie stanu użytkownika

Cechy	Context	Redux Toolkit	Zustand	Jotai	MobX
Najlepsze dla	konfiguracja, motyw, user	większy client state	prosty store	atomowy stan	model reaktywny
Boilerplate	Niskie	Średnie	Niskie	Niskie	Średnie
Selektory	Nie natywnie	Tak	Tak	Tak	Tak
DevTools	React DevTools	Redux DevTools	Redux DevTools przez middleware	ograniczone	MobX DevTools
Krzywa wejścia	Niska	Średnia	Niska	Średnia	Średnia
Kiedy uważać	częste zmiany wartości	małe aplikacje	duże zespoły bez konwencji	niszowy ekosystem	niejawne zależności

21.10.2026 19:17

Meta-Frameworki oparte o React

Meta-frameworki rozszerzają React o routing, ładowanie i mutowanie danych, code splitting, renderowanie serwerowe lub statyczne oraz konwencje wdrożeniowe.

Przykłady:

- Next.js
- Remix
- Gatsby
- Expo (z React Native)
- Blitz.js

Jak wybierać framework React?

Rozwiązanie	Typowy kontekst
React Router Framework Mode	aplikacje SPA, SSR lub SSG z route modules, loaderami i actions
Next.js	aplikacje full-stack wykorzystujące Server Components, Server Functions i wiele strategii renderowania
Remix	full-stack oparty o web platform; przed wyborem sprawdź relację używanej wersji z React Router
Gatsby	projekty contentowe wykorzystujące SSG, DSG lub SSR
Expo	aplikacje React Native i uniwersalne aplikacje mobilne
Blitz.js	projekty korzystające z jego konwencji full-stack i integracji z ekosystemem Next.js

Tabela możliwości szybko się starzeje

- sprawdzaj aktualną dokumentację wersji przed wyborem frameworka
- porównuj wymagany model hostingu, renderowania, danych i wdrożenia
- Server Components i Server Functions wymagają integracji zapewnianej przez framework lub toolchain
- możliwość użycia SSR nie oznacza, że każda trasa musi być renderowana na serwerze

React Server Components

- Server Components są renderowane w środowisku serwerowym lub podczas builda
- ich kod nie jest wysyłany do przeglądarki jako część bundla klienta
- mogą bezpośrednio odczytywać dane i korzystać z zależności dostępnych wyłącznie po stronie serwera
- nie mogą używać stanu, efektów ani API przeglądarki
- ich obsługa wymaga frameworka lub toolchainu implementującego protokół React Server Components

Server Component a SSR

Server Components	SSR
opisują granicę wykonywania komponentów	opisuje sposób wygenerowania początkowego HTML
kod komponentu serwerowego nie trafia do bundla klienta	komponent może zostać później uwodniony po stronie klienta
mogą działać podczas builda albo na serwerze	zwykle działa w odpowiedzi na żądanie lub podczas prerenderingu

Te mechanizmy mogą być używane razem, ale rozwiązują inne problemy.

Granica "use client"

Przykładowy kod:

```tsx
// LikeButton.tsx
"use client";

import { useState } from 'react';

export function LikeButton() {
  const [liked, setLiked] = useState(false);

  return (
    <button onClick={() => setLiked((value) => !value)}>
      {liked ? 'Lubisz to' : 'Polub'}
    </button>
  );
}
```

- `"use client"` wyznacza granicę modułów wykonywanych po stronie klienta
- props przekazywane przez granicę muszą być serializowalne
- dyrektywy nie dodajemy automatycznie do każdego komponentu

React Server Functions

- Server Function jest funkcją wykonywaną po stronie serwera, do której klient otrzymuje bezpieczną referencję
- dyrektywa `"use server"` oznacza funkcję lub moduł jako serwerowy
- funkcję można przekazać między innymi do `action` formularza
- framework odpowiada za transport argumentów i wyniku pomiędzy klientem a serwerem
- każda Server Function nadal musi samodzielnie sprawdzać dane, uprawnienia i uwierzytelnienie

Przykładowy kod:

```
// actions.ts
"use server";

export async function saveBook(formData: FormData) {
  const title = String(formData.get('title') ?? '').trim();

  if (!title) {
    return { error: 'Tytuł jest wymagany' };
  }

  await booksRepository.create({ title });
  return { error: null };
}
```

Przykładowy kod:

```
"use client";

import { useActionState } from 'react';
import { saveBook } from './actions';

const initialState = { error: null };

export function BookForm() {
  const [state, submitAction, isPending] = useActionState(
    (_state, formData) => saveBook(formData),
    initialState
  );

  return (
    <form action={submitAction}>
      <input name="title" />
      <button disabled={isPending}>Zapisz</button>
      {state.error && <p role="alert">{state.error}</p>}
    </form>
  );
}
```

Server Function, endpoint czy funkcja klienta?

Mechanizm	Kiedy użyć
Server Function	operacja jest częścią frameworkowego przepływu danych Reacta
endpoint HTTP	potrzebujemy publicznego lub niezależnego kontraktu API
funkcja klienta	operacja nie wymaga zaufanego środowiska serwerowego

„Server Action” to często używana nazwa Server Function przekazanej jako akcja formularza lub komponentu.

useOptimistic

- `useOptimistic` pozwala natychmiast pokazać przewidywany wynik operacji asynchronicznej
- stan bazowy pozostaje źródłem prawdy
- stan optymistyczny obowiązuje podczas trwania Action lub Transition
- po zakończeniu operacji React ponownie wyprowadza widok ze stanu bazowego
- interfejs powinien jasno pokazywać element oczekujący i obsłużyć błąd operacji

Przykładowy kod:

```
const [optimisticBooks, addOptimisticBook] = useOptimistic(
  books,
  (currentBooks, newBook: Book) => [
    ...currentBooks,
    { ...newBook, pending: true },
  ]
);

// Funkcja jest przekazana do prop `action` formularza.
async function addBookAction(formData: FormData) {
  const book = createBookFrom(formData);
  addOptimisticBook(book);
  await saveBook(book);
}

return <form action={addBookAction}>...</form>;
```

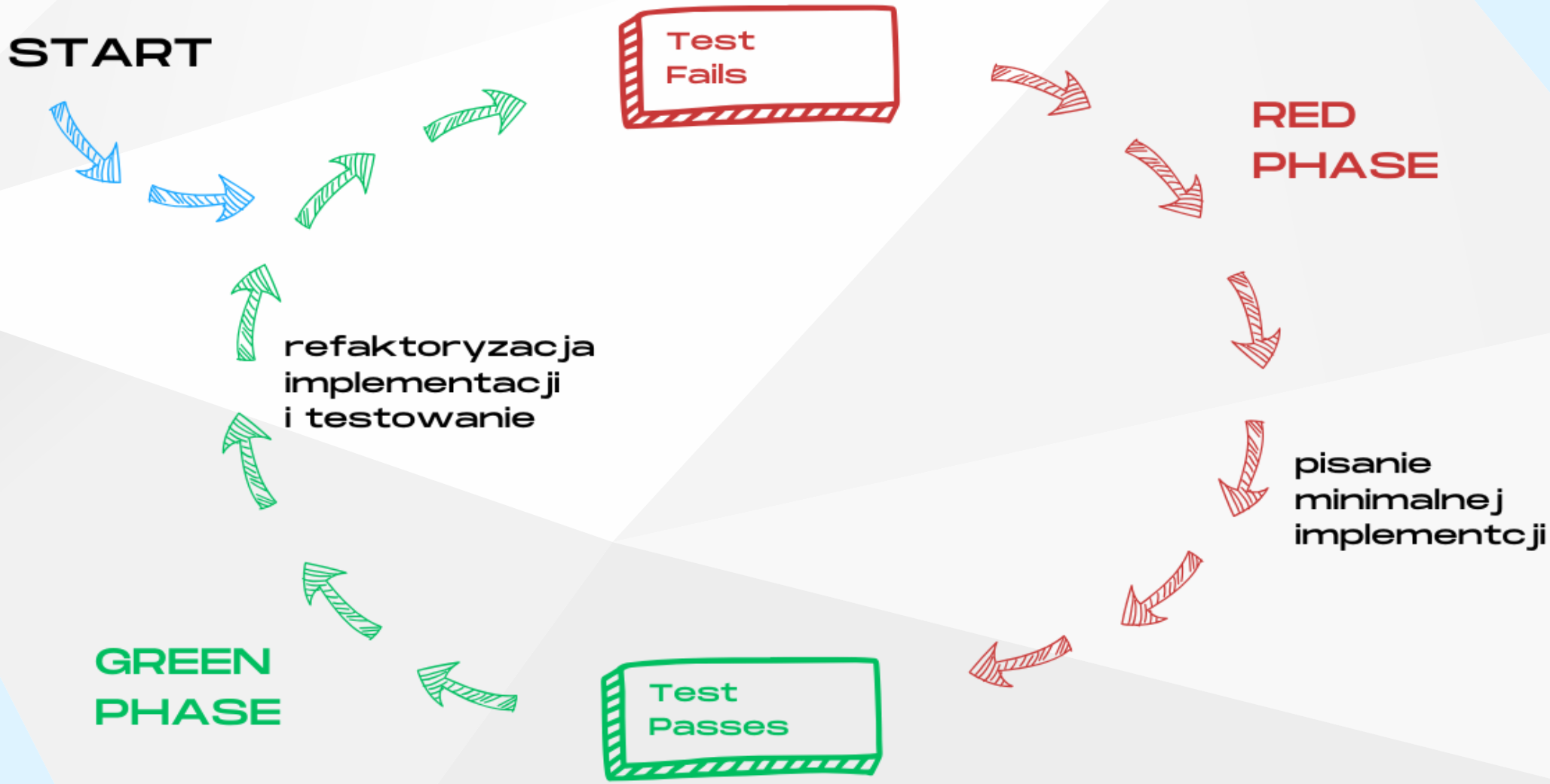
useOptimistic a TanStack Query

`useOptimistic`	TanStack Query
lokalny optymistyczny widok komponentu	optymistyczna aktualizacja współdzielonego cache
dobrze współpracuje z Actions	dobrze współpracuje z mutacjami API
nie zapewnia cache ani invalidacji	zarządza cache, invalidacją i ponownym pobieraniem

Wybór zależy od tego, czy optymistyczna zmiana jest lokalnym stanem interfejsu, czy częścią współdzielonego server state.

TDD

- Test Driven Development
- testy bazują na cyklu: **Red / Green / Refactoring**
  - **Red** — w tej fazie myślimy o tym co chcemy zaprogramować
  - **Green** — ta faza pokrywa odpowiedź na pytanie jak zrobić, żeby przeszły testy
  - **Refactor** — myślimy o tym jak poprawić napisaną w poprzednich fazach implementację
- w tym podejściu do testowania testy piszemy zanim zostanie napisana rzeczywista implementacja


START
Test
Fails
RED
PHASE
pisanie
minimalnej
implementcji
Test
Passes
GREEN
PHASE
refaktoryzacja
implementacji
i testowanie

TDD: Red

- czerwona faza to zawsze punkt wyjściowy od którego zaczynamy testowanie
- celem tej fazy jest napisanie testów, które informują nas o tym co powinna robić implementacja funkcjonalności (definiują cel funkcjonalności)
- na końcu tej fazy wszystkie testy powinny być czerwone (funkcjonalność jeszcze nie istnieje)
- testy będą przechodzić dopiero kiedy wymagania zostaną pokryte

TDD: Green

- w zielonej fazie piszemy kod implementacji, tak aby testy napisane w fazie czerwonej przeszły (stały się zielone)
- celem jest znalezienie rozwiązania, bez głębszego wchodzenia w optymalizację naszej implementacji
- na końcu tej fazy „jesteśmy na zielono”. Możemy zacząć myśleć o optymalizacjach naszego kodu

TDD: Refaktoryzacja

- w tej fazie nadal „jesteśmy na zielono”
- ostatnia faza to moment na zastanowienie się jak napisać nasz kod lepiej lub bardziej wydajnie
- podczas refaktoryzacji nie chodzi o zmiany wyniku naszej funkcji, a o osiągnięcie tego samego efektu przy bardziej opisowym lub szybszym kodzie

Pytania, które powinniśmy sobie zadać kiedy myślimy o refaktoryzacji:

- Czy mogę poprawić moje testy, aby były lepiej dopasowane?
- Czy moje testy zwracają wartościowy feedback na temat funkcjonalności?
- Czy moje testy są odpowiednio odizolowane od innych testów / modułów?
- Czy mogę zredukować duplikację kodu w moich testach lub w samej implementacji?
- Czy moja implementacja może być czytelniejsza?
- Czy moja implementacja może być bardziej wydajna?

White Box

- testy strukturalne
- testy bazują na kodzie źródłowym
- o wiele łatwiej przygotować kod do całościowej optymalizacji dzięki testom
- łatwiejsze do automatyzacji

Black Box

- testy funkcjonalne
- nie mamy dostępu do kodu źródłowego
- testujemy tylko zakładane funkcjonalności
- w tego typu testach mamy większą szansę na znalezienie większej liczby błędów, ale niekonicznie ich pochodzenia

Grey Box

- testy hybrydowe, łączące Black Box i White Box
- tester ma ograniczoną wiedzę o kodzie źródłowym, np. zna architekturę systemu, ale nie ma pełnego dostępu do kodu
- umożliwia bardziej efektywne testowanie, gdyż pozwala na lepsze zrozumienie potencjalnych słabych punktów aplikacji
- często stosowane w testach bezpieczeństwa oraz testach integracyjnych

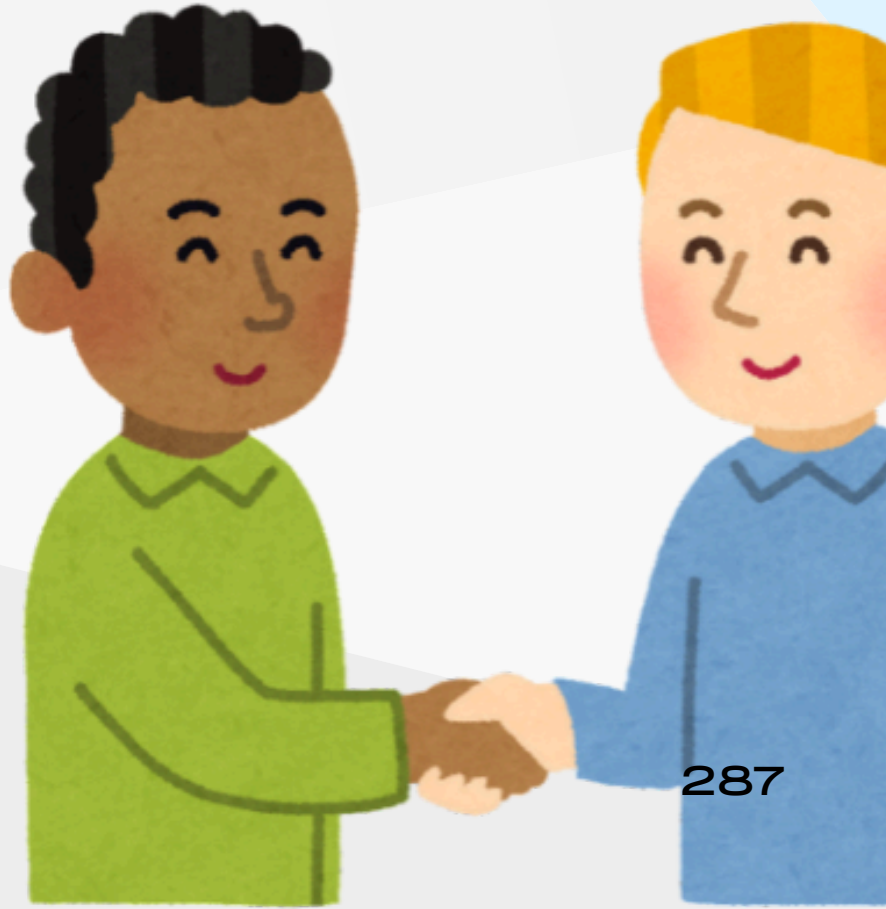
Antywzorce w testach

Obywatel drugiej kategorii

- kod testów traktowany jako ten gorszy
- nie jest utrzymywany, wszędzie mamy do czynienia z duplikacją kodu, chaosem i mylącymi nazwami

Brak szacunku

- deweloper robi jakąś zmianę i nie aktualizuje unit testów
- po wprowadzeniu system continuous integration zwraca błąd spowodowany nieprzechodzącymi testami
- zamiast poprawić testy usuwa się je, zakomentowuje albo ignoruje



Optymistyczna ścieżka

- testowanie tylko podstawowego działania funkcji bez zastanowienia się nad możliwymi wyjątkami, warunkami brzegowymi, czy złośliwymi kombinacjami danych wejściowych.

Gigant

- test zawierający bardzo dużo linii kodu – kilkaset, albo nawet ponad tysiąc
- test jest tak duży, że nie wiadomo do końca co robi, jego utrzymanie jest bardzo trudne i może on posiadać swoje własne bugi

Pasażer na gapę

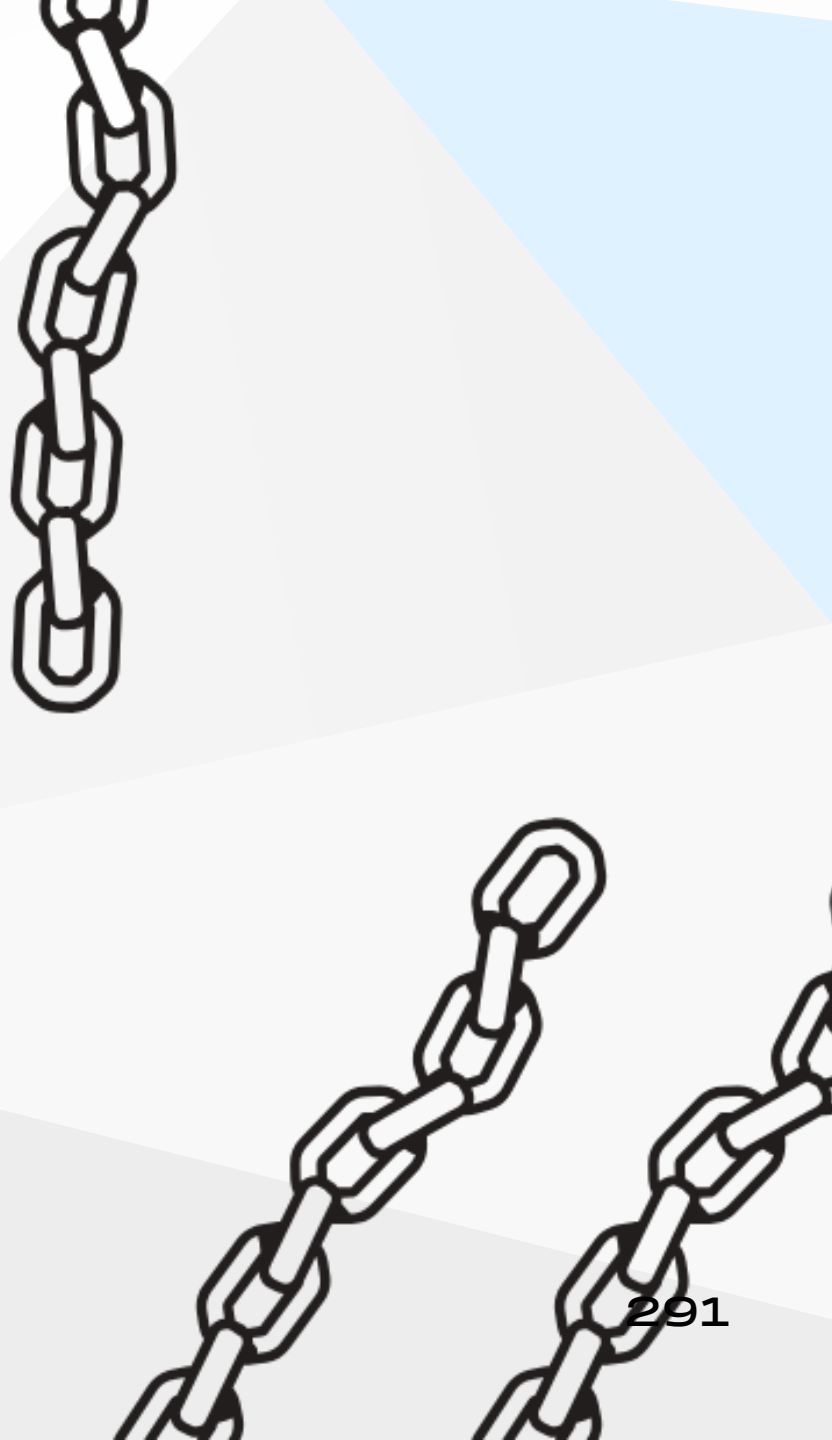
- zamiast stworzyć nowy test, dodajemy kolejny cykl Arrange-Act-Assert do istniejącego testu
- w ten sposób istniejące testy się wydłużają, a ich cel się rozmywa
- w końcu taki test przeobraża się w giganta

Przykładowy kod:

```
test("Zbyt długi test – pasażer na gapę", () => {
  const user = createUser("Bob", 25);
  expect(user.name).toBe("Bob");

  user.updateAge(26);
  expect(user.age).toBe(26);

  user.deactivate();
  expect(user.isActive).toBe(false);
});
```

Chain gang

- kolejne testy są od siebie zależne i muszą być wykonywane w odpowiedniej kolejności
- usunięcie lub zmiana jednego testu z łańcucha skutkuje wtedy zepsuciem całej grupy testów



Sobowtór

- na potrzeby testu mockujemy jakąś zależność, jednak aby wykonać test nie wystarczy nam podstawić zwracaną wartość - zamiast tego w mocku kopiujemy zachowanie rzeczywistej funkcji
- jeżeli będziemy chcieli wprowadzić zmiany, musimy je uwzględnić zarówno w kodzie produkcyjnym, jak i w mocku

Kłamca

- test jest tak skonstruowany, żeby wchodził w odpowiednie ścieżki w kodzie i nabijał w ten sposób code coverage
- jednak nie sprawdza on w żaden sposób poprawności wykonywania tych ścieżek

Inspektor

- aby osiągnąć lepsze pokrycie, test bazuje na konkretnej implementacji i łamie zasady enkapsulacji
- w ten sposób otrzymujemy skomplikowane testy utrudniające refactor kodu w przyszłości

Flaky test

- test raz przechodzi, a raz nie – bez zmiany kodu
- może być zależny od kolejności uruchamiania, środowiska czy nawet czasu
- powoduje frustrację i zmniejsza zaufanie do testów

Przykładowy kod:

```
test("Flaky test - zależny od czasu", () => {
  const now = new Date().getSeconds();
  expect(now % 2).toBe(0); // Test przejdzie tylko w parzystych sekundach
});
```

Mr. Tester

- test, który testuje wszystko naraz
- zamiast jednego konkretnego przypadku sprawdza kilka rzeczy jednocześnie
- trudny do zrozumienia i poprawienia, bo nie wiadomo, co dokładnie się zepsuło

Przykładowy kod:

```
test("Sprawdza wszystko na raz", () => {
  const user = createUser("Alice", 30);

  expect(user.name).toBe("Alice");
  expect(user.age).toBe(30);
  expect(user.isActive).toBe(true);
  expect(user.permissions.length).toBeGreaterThan(0);
});
```

Śpiący test

- używa `sleep()` lub innych sztucznych opóźnień, zamiast prawidłowego oczekiwania na zakończenie operacji asynchronicznych
- spowalnia cały proces testowania
- czasami testy przechodzą, a czasami nie, zależnie od tego, jak wolno działa system

Przykładowy kod:

```
test("Sleepy test", (done) => {
  setTimeout(() => {
    expect(true).toBe(true);
    done();
  }, 5000); // Sztuczne czekanie 5 sekund!
});
```

Martwy kod testowy

- testy, których nikt nie uruchamia
- testy, które są zawsze wykomentowane lub ignorowane
- testy, które nie mogą się nie powieść (np. `assert(true)`)

Co powinniśmy testować na FE?

jednostkowo

- pojedyncze funkcje i komponenty w izolacji
- logikę biznesową (np. walidację formularzy, obliczenia, konwersje)
- czyste funkcje pomocnicze i reguły przekształcania danych

integracyjnie

- kluczowe ścieżki aplikacji (np. logowanie, wyszukiwanie, formularze)
- rzeczywiste interakcje użytkownika z komponentami
- komunikację z API mockowaną na poziomie HTTP, np. przez MSW

e2e

- kluczowe ścieżki użytkownika
- rzeczywiste interakcje w przeglądarce
- krytyczne integracje, których nie da się wiarygodnie sprawdzić w jsdom

WAŻNE

Domyślne pliki testów

- `*.test.ts`
- `*.test.tsx`
- `*.spec.ts`
- `*.spec.tsx`
- katalog `__tests__`, jeśli projekt ma już taką konwencję

Vitest / Jest

- runner uruchamia testy, udostępnia `test`, `expect`, mocki, timery i tryb watch
- w projektach opartych o Vite naturalnym wyborem jest Vitest
- Jest nadal często występuje w istniejących projektach i w wielu bibliotekach
- testy komponentów React najczęściej uruchamiamy w środowisku `jsdom`
- `jsdom` symuluje DOM, ale nie jest pełną przeglądarką

Konfiguracja Vitest + Testing Library

Przykładowy kod:

```
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

Przykładowy kod:

```
// vite.config.ts
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vitest/config';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: './src/test/setup.ts',
  },
});
```


21.10.2026 19:17

Przykładowy kod:

```
// src/test/setup.ts
import '@testing-library/jest-dom/vitest';
```

Przykładowy kod:

```
{
  "scripts": {
    "test": "vitest",
    "test:run": "vitest run"
  }
}
```

Matchery

Vitest i Jest udostępniają matchery, aby ułatwić porównywanie wartości na wiele różnych sposobów.

Matcher	Opis
toBe	używa Object.is do testowania dopasowania ścisłego
toEqual	rekursywnie sprawdza każde pole w obiekcie lub tablicy
not	
toBeNull, toBeUndefined, toBeDefined, toBeTruthy, toBeFalsy	dla sprawdzenia prawdziwości

Matcher	Opis
toBeGreaterThan, toBeLessThan, toBeGreaterThanOrEqual, toBeLessThanOrEqual, toBeCloseTo	dla liczb
toMatch	dla stringów
toContain	dla tablic
toThrow	dla wyjątków

Przykładowy kod:

```
import { expect, test } from 'vitest';

test('two plus two is four', () => {
  expect(2 + 2).toBe(4);
});

test('object assignment', () => {
  const data = {one: 1};
  data['two'] = 2;
  expect(data).toEqual({one: 1, two: 2});
});

test('adding positive numbers is not zero', () => {
  for (let a = 1; a < 10; a++) {
    for (let b = 1; b < 10; b++) {
      expect(a + b).not.toBe(0);
    }
  }
});
```

Async

- kod JavaScript uruchamiany jest zazwyczaj asynchronicznie (zdarzenia, zapytania itp)
- zwrócenie obietnicy z testu spowoduje, że runner będzie czekał aż obietnica zostanie rozwiązana
- Vitest i Jest posiadają matchery do kodu asynchronicznego: `resolves` i `rejects`

Przykładowy kod:

```
import { expect, test } from 'vitest';

test('the data is peanut butter', () => {
  return fetchData().then(data => {
    expect(data).toBe('peanut butter');
  });
});

test('the data is peanut butter', async () => {
  const data = await fetchData();
  expect(data).toBe('peanut butter');
});

test('the fetch fails with an error', async () => {
  expect.assertions(1);
  try {
    await fetchData();
  } catch (e) {
    expect(e).toMatch('error');
  }
});

test('the data is peanut butter', async () => {
  await expect(fetchData()).resolves.toBe('peanut butter');
});
```

Przykładowy kod:

```
import { describe, expect, test } from 'vitest';
import { sum } from './sum';

describe('sum module', () => {
  describe('when numbers are passed', () => {
    test('adds 1 + 2 to equal 3', () => {
      expect(sum(1, 2)).toBe(3);
    });
  });
});
```

Setup testów

- jeśli musimy wykonać operacje wielokrotnie przed i po uruchomieniu jakiegoś testu, możemy skorzystać z Hooków: `beforeEach` i `afterEach`.
- w niektórych przypadkach potrzebujesz wykonać jakąś operację raz przed / po wszystkimi testami z danej grupy, dla takich przypadków runner udostępnia hooki: `beforeAll` i `afterAll`
- jeśli musimy uruchomić tylko jeden test (ponieważ nam nie przechodzi), tymczasowo możemy zastąpić funkcję `test` za pomocą funkcji `test.only`

Przykładowy kod:

```
import { beforeEach, expect, test } from 'vitest';

beforeEach(() => {
  return initializeCityDatabase();
});

test('city database has Vienna', () => {
  expect(isCity('Vienna')).toBeTruthy();
});
```

Przykładowy kod:

```
import { expect, test } from 'vitest';

test.only('this will be the only test that runs', () => {
  expect(true).toBe(false);
});

test('this test will not run', () => {
  expect('A').toBe('A');
});
```

Mockowanie funkcji

- mockowanie funkcji zezwala na testowanie połączeń pomiędzy różnymi częściami kodu za pomocą usunięcia aktualnej implementacji funkcji, przechwyceniem jej uruchomienia (wraz z parametrami), przychwycenie instancji konstruktora funkcji, kiedy zostały zainicjowane przez operator `new`
- pozwala ponadto na zwrócenie własnej wartości z mockowanej funkcji
- wszystkie mockowane funkcje mają specjalną właściwość `mock`, gdzie możemy znaleźć informacje na temat ilości i sposobu wywołania danej funkcji
- możemy wykorzystać mockowanie implementacji całych modułów
- po zakończeniu testu warto wywołać `mockRestore()` na mocku, aby zapobiec przeciekaniu mocków pomiędzy testami

Przykładowy kod:

```
import { vi } from 'vitest';

function forEach(items, callback) {
  for (let index = 0; index < items.length; index++) {
    callback(items[index]);
  }
}
const mockCallback = vi.fn(x => 42 + x);
forEach([0, 1], mockCallback);
expect(mockCallback.mock.calls.length).toBe(2);
```


mateuszjablonski.com × Sages
21.10.2026 19:17

Mockowanie API

- nie mockujemy `fetch` albo `axios` w każdym teście komponentu, jeśli test ma sprawdzać zachowanie widoku
- dla komponentów komunikujących się z backendem lepszym domyślnym wyborem jest mockowanie na poziomie HTTP
- MSW pozwala uruchomić test z tym samym kodem requestu, którego używa aplikacja

Przykładowy kod:

```
import { http, HttpResponse } from 'msw';
import { render, screen } from '@testing-library/react';
import { expect, test } from 'vitest';
import { server } from './testServer';
import { Readers } from './Readers';

test('shows readers loaded from API', async () => {
  server.use(
    http.get('/api/readers', () => {
      return HttpResponse.json([{ id: '1', name: 'Ada' }]);
    })
  );

  render(<Readers />);

  expect(
    await screen.findByRole('heading', { name: /ada/i })
  ).toBeInTheDocument();
});
```

Timers

- natywne funkcje timerów (np. `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`) są mniej niż idealne dla testowania na środowiskach testerskich, ponieważ są zależne od prawdziwego upływu czasu
- w naszym teście możemy włączyć `fake timers` poprzez wywołanie `vi.useFakeTimers()` albo `jest.useFakeTimers()`

Przykładowy kod:

```
import { vi } from 'vitest';
import { timerGame } from '../timerGame';

vi.useFakeTimers();
vi.spyOn(global, 'setTimeout');

test('waits 1 second before ending the game', () => {
  timerGame();

  expect(setTimeout).toHaveBeenCalledTimes(1);
  expect(setTimeout).toHaveBeenLastCalledWith(expect.any(Function), 1000);
});
```

Optymalizacja testów

Testy równoległe

Runner może uruchamiać testy równolegle, co skraca czas trwania testów w dużych projektach. Liczbę workerów warto kontrolować w CI, gdy testy są pamięciożerne.

Przykładowy kod:

```
vitest --maxWorkers=4
```

Testowanie tylko zmienionych plików

Tryb watch pozwala uruchamiać testy po zmianie pliku i szybciej pracować lokalnie.

Przykładowy kod:

```
vitest --watch
```

Testing Library

- biblioteka do testowania UI przez zachowanie widoczne dla użytkownika
- nie zastępuje runnera testów - działa z Vitestem, Jestem i innymi runnerami
- zachęca do wyszukiwania elementów po roli, etykiecie, tekście i dostępnej nazwie
- ogranicza testowanie szczegółów implementacyjnych komponentu
- `userEvent` lepiej modeluje interakcje użytkownika niż ręczne wywoływanie eventów

Przykładowy kod:

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { expect, test } from 'vitest';
import { BookSearch } from './BookSearch';

test('filters books by title', async () => {
  const user = userEvent.setup();

  render(<BookSearch />);

  await user.type(
    screen.getByRole('textbox', { name: /szukaj/i }),
    'React'
  );

  expect(
    screen.getByRole('link', { name: /react w praktyce/i })
  ).toBeInTheDocument();
  expect(screen.queryByText(/vue w praktyce/i)).not.toBeInTheDocument();
});
```

Selektory w Testing Library

Typ	Kiedy używać
`getByRole`	domyślny wybór dla interaktywnych i semantycznych elementów
`getByLabelText`	pola formularzy z etykietą
`getByText`	statyczna treść, której użytkownik szuka wzrokiem
`getByTestId`	ostatnia deska ratunku dla elementów bez semantyki

21.10.2026 19:17

Query	Brak dopasowania	1 dopasowanie	Więcej dopasowań	Await?
getBy	throw	return	throw	brak
findBy	throw	return	throw	tak
queryBy	null	return	throw	brak
getAllBy	throw	array	array	brak
findAllBy	throw	array	array	tak
queryAllBy	[]	array	array	brak

Zadanie nr 24

25 minut

- zainstaluj Vitest, Testing Library, `@testing-library/jest-dom` , `@testing-library/user-event` oraz `jsdom`

Przykładowy kod:

```
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

- skonfiguruj Vitest w `vite.config.ts`
- dodaj skrypty `test` oraz `test:run` w `package.json`
- przetestuj jedną funkcję z `utils/`
- przetestuj jeden komponent przez zachowanie widoczne dla użytkownika
- użyj `userEvent` do interakcji zamiast ręcznie wywoływać eventy
- jeśli komponent pobiera dane, zainstaluj MSW i mockuj odpowiedź HTTP przez handler

Przykładowy kod:

```
npm install -D msw
```

Pytania po zadaniu

PYTANIE

Które asercje opisują zachowanie widoczne dla użytkownika?

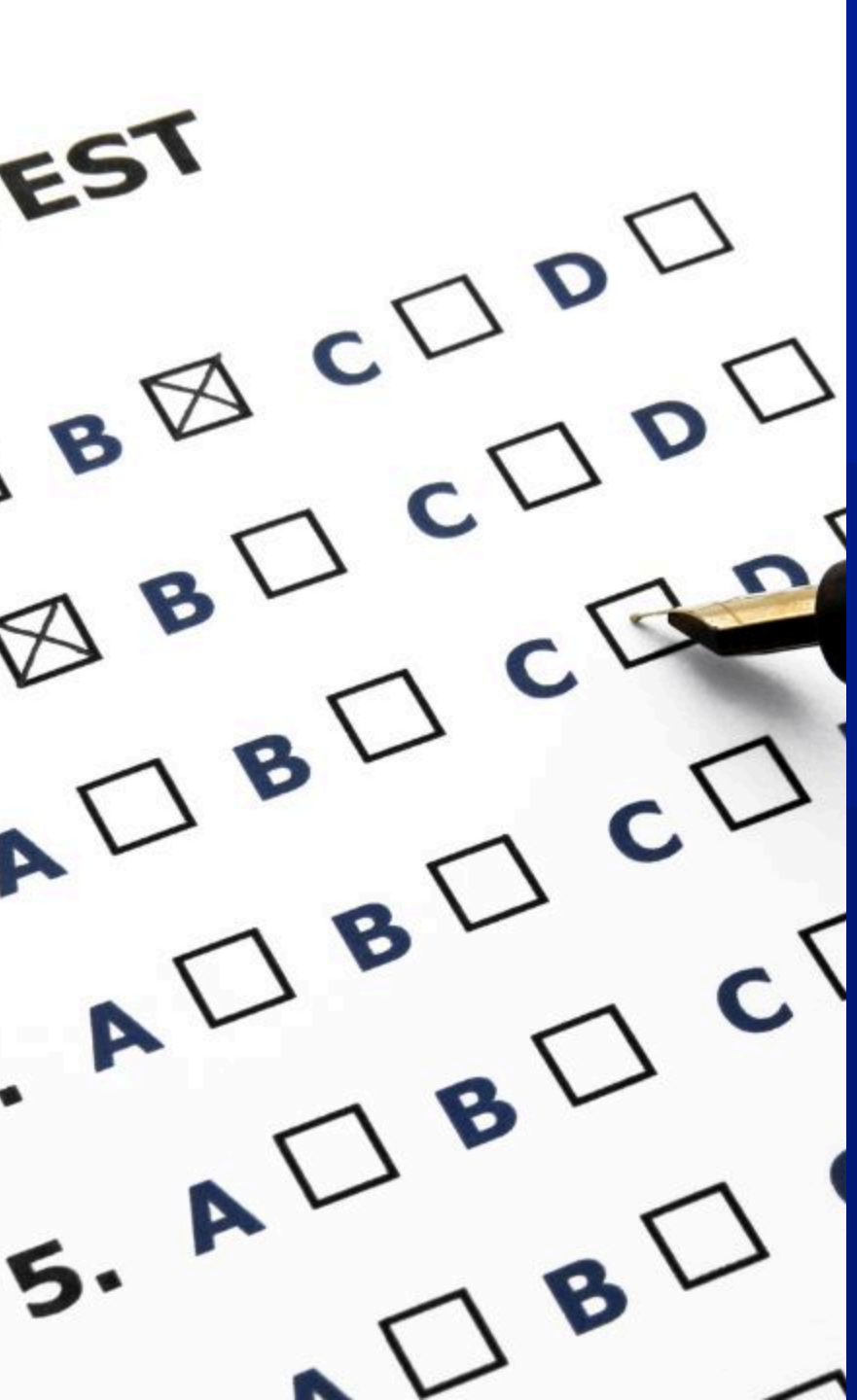
PYTANIE

Gdzie powinna przebiegać granica mockowania komunikacji HTTP?

PYTANIE

Którego istotnego ryzyka nadal nie obejmują przechodzące testy?

Dodatkowe pytania?



KAHOOT

Dołącz na **kahoot.it**
lub przez aplikację **Kahoot!**

Ankieta

czyli jak mi poszło?

https://sages.link/890915

Dziękuję za uwagę

Mateusz Jabłoński

mail@mateuszjablonski.com
mateuszjablonski.com

21.10.2026 19:17